

PROYECTO

Nube con Servidor Dedicado Low Cost

Guía Definitiva — Infraestructura, Operación, Monitoreo y Mantenimiento

Stack: Nextcloud + Docker + Cloudflare Tunnel + Tailscale + MCP

Versión 3.3 — Junio 2026

Costo anual: ~\$46 USD

Guía Definitiva — Proyecto Nube con Servidor Dedicado Low Cost

Versión 3.3 — Junio 2026 **Proyecto:** Nube Personal con Servidor Dedicado Low Cost **Stack:** Nextcloud + Docker + Cloudflare Tunnel + Tailscale + MCP de gestión

Esta guía integra el proyecto original, todas las mejoras aplicadas, los incidentes resueltos, los procedimientos de diagnóstico y recuperación, el sistema de monitoreo en capas, el servidor MCP de gestión asistida por IA, y un catálogo completo de comandos. Diseñada para **aplicar**, **estudiar** y **orientar**.

Novedades v3.3: Fase 13 (MCP de gestión del NUC vía Claude) · Fase 14 (diagnóstico y resolución de inestabilidad WiFi: fix de driver + cambio de punto de acceso, con evidencia de 24 h) · Fase 15 (actualización de Nextcloud 33.0.2→33.0.4 con fijado de imagen Docker).

Tabla de contenidos

Parte 1 — Fundamentos del Proyecto

1. Introducción
2. Conceptos fundamentales y glosario
3. Disciplinas aplicadas
4. Arquitectura del proyecto

Parte 2 — Implementación

5. Desarrollo paso a paso (Fases 1-7)
6. Archivos de configuración
7. Infrastructure as Code
8. Costos del proyecto

Parte 3 — Evolución y Mejoras Aplicadas

9. Fase 8: Incidente y recuperación de datos
10. Fase 9: Backup off-site (Google Drive)
11. Fase 10: Notify Push (sincronización instantánea)
12. Fase 11: Diagnóstico forense del sistema
13. Fase 12: Sistema de monitoreo en capas 13b. Fase 13: MCP de gestión del NUC (Claude) 13c. Fase 14: Inestabilidad WiFi y degradación del túnel 13d. Fase 15: Actualización de Nextcloud y fijado de imagen

Parte 4 — Operación del Sistema

- 14. Diagnóstico del sistema (qué, cómo, cuándo)
- 15. Recuperación ante desastres
- 16. Mantenimiento periódico
- 17. Troubleshooting de errores comunes

Parte 5 — Referencia

- 18. Catálogo completo de comandos
 - 19. Recomendaciones finales
 - 20. Estado final del sistema
-

Parte 1 — Fundamentos del Proyecto

1. Introducción

1.1 ¿Qué es una nube personal?

Una nube personal es un servicio de almacenamiento y sincronización de archivos que funciona en un servidor propio, en lugar de depender de servidores de terceros como Google, Microsoft o Apple. Permite guardar documentos, fotos, vídeos y cualquier tipo de archivo, accediendo a ellos desde cualquier dispositivo conectado a internet.

La principal ventaja: **control total sobre los datos**.

1.2 Objetivo

Implementar un servicio de almacenamiento en la nube privado, seguro y accesible desde cualquier lugar, con tecnologías open-source y un servidor de bajo consumo (NUC), con costo operativo mínimo.

1.3 Comparación con servicios comerciales

Característica	Google Drive	Dropbox	Nube Personal
Almacenamiento	15 GB / 2 TB (\$10/mes)	2 GB / 2 TB (\$12/mes)	1.7 TB (disco propio)
Costo anual	\$120 USD (2 TB)	\$144 USD (2 TB)	~\$46 USD
Privacidad	Google accede	Dropbox accede	Control total
Usuarios	Limitado	Limitado	Ilimitados
Personalización	Mínima	Mínima	Total

2. Conceptos fundamentales y glosario

2.1 Redes y comunicaciones

- **IP (Internet Protocol):** Dirección numérica única (ej: 192.168.1.69)
- **DNS (Domain Name System):** Traduce nombres a IPs (cloud.hexa38.com → IP)
- **HTTPS/SSL/TLS:** Cifrado de comunicaciones web
- **Puerto:** Punto de entrada/salida (80=HTTP, 443=HTTPS, 22=SSH)
- **Firewall (UFW):** Filtra tráfico de red
- **SSH (Secure Shell):** Acceso remoto cifrado

- **Tunnel:** Conexión cifrada sin abrir puertos (Cloudflare Tunnel)
- **HSTS:** Fuerza siempre HTTPS
- **Reverse Proxy:** Servidor que recibe peticiones y las redirige a servicios internos
- **WebSocket (ws://):** Conexión bidireccional en tiempo real

2.2 Sistemas operativos y servidores

- **Linux (Ubuntu Server):** SO open-source para servidores
- **NUC (Next Unit of Computing):** Mini PC Intel de bajo consumo
- **Swap:** Memoria virtual en disco (extiende RAM)
- **ext4:** Sistema de archivos Linux con journaling
- **NTFS:** Sistema de archivos Windows
- **Montaje de discos:** Hacer un disco accesible al SO
- **fstab:** Archivo que define discos montados al inicio
- **systemd:** Sistema de inicio y gestión de servicios

2.3 Contenedores y virtualización

- **Docker:** Plataforma de contenedores
- **Contenedor:** Entorno aislado y ligero (comparte kernel del host)
- **Imagen:** Plantilla de solo lectura
- **Docker Compose:** Define múltiples contenedores en un YAML
- **Volumen:** Almacenamiento persistente
- **Bind mount:** Carpeta del host mapeada al contenedor
- **Red bridge:** Red virtual entre contenedores

2.4 Base de datos

- **MariaDB:** SGBD relacional (fork de MySQL)
- **SQL:** Lenguaje de consultas
- **Dump:** Exportación completa de la BD a archivo
- **InnoDB:** Motor de almacenamiento transaccional

2.5 Cache y rendimiento

- **Redis:** Almacén en memoria para cache
- **APCu:** Cache local PHP en memoria
- **OPcache:** Cache de bytecode PHP

2.6 Seguridad

- **2FA/TOTP:** Autenticación de dos factores temporales
- **WHOIS:** Registro público de dueños de dominios

- **Contraseña de aplicación:** Credencial específica para apps externas
- **Fuerza bruta:** Ataque por prueba sistemática
- **Rate limiting:** Limita peticiones por tiempo
- **fail2ban:** Banea IPs con intentos fallidos

2.7 Dominio y DNS

- **Dominio:** Nombre único en internet (hexa38.com)
- **Subdominio:** Extensión (cloud.hexa38.com)
- **Cloudflare:** DNS, CDN, túneles, SSL
- **Nameservers:** Servidores que resuelven dominios
- **Cloudflare Tunnel:** Conexión cifrada sin IP pública

2.8 Automatización

- **Cron/Crontab:** Programador de tareas en Linux
- **Script:** Archivo con instrucciones secuenciales
- **Backup:** Copia de seguridad
- **rclone:** Herramienta de sincronización con clouds (Drive, S3, etc.)
- **IaC:** Infrastructure as Code

2.9 SMART y discos (añadido v2)

- **SMART:** Self-Monitoring, Analysis and Reporting Technology
- **Reallocated_Sector_Ct:** Sectores defectuosos reasignados
- **Current_Pending_Sector:** Sectores pendientes de reasignar
- **Power_On_Hours:** Horas que el disco lleva encendido
- **Load_Cycle_Count:** Veces que el cabezal se aparcó (HDD)
- **Wear_Leveling_Count:** Desgaste del SSD

2.10 Push notifications (añadido v2)

- **notify_push:** Servidor de Nextcloud para sync instantáneo
 - **WebSocket:** Conexión persistente bidireccional
 - **trusted_proxies:** IPs en las que Nextcloud confía
 - **forwarded_for_headers:** Headers de IP real del cliente
-

3. Disciplinas aplicadas

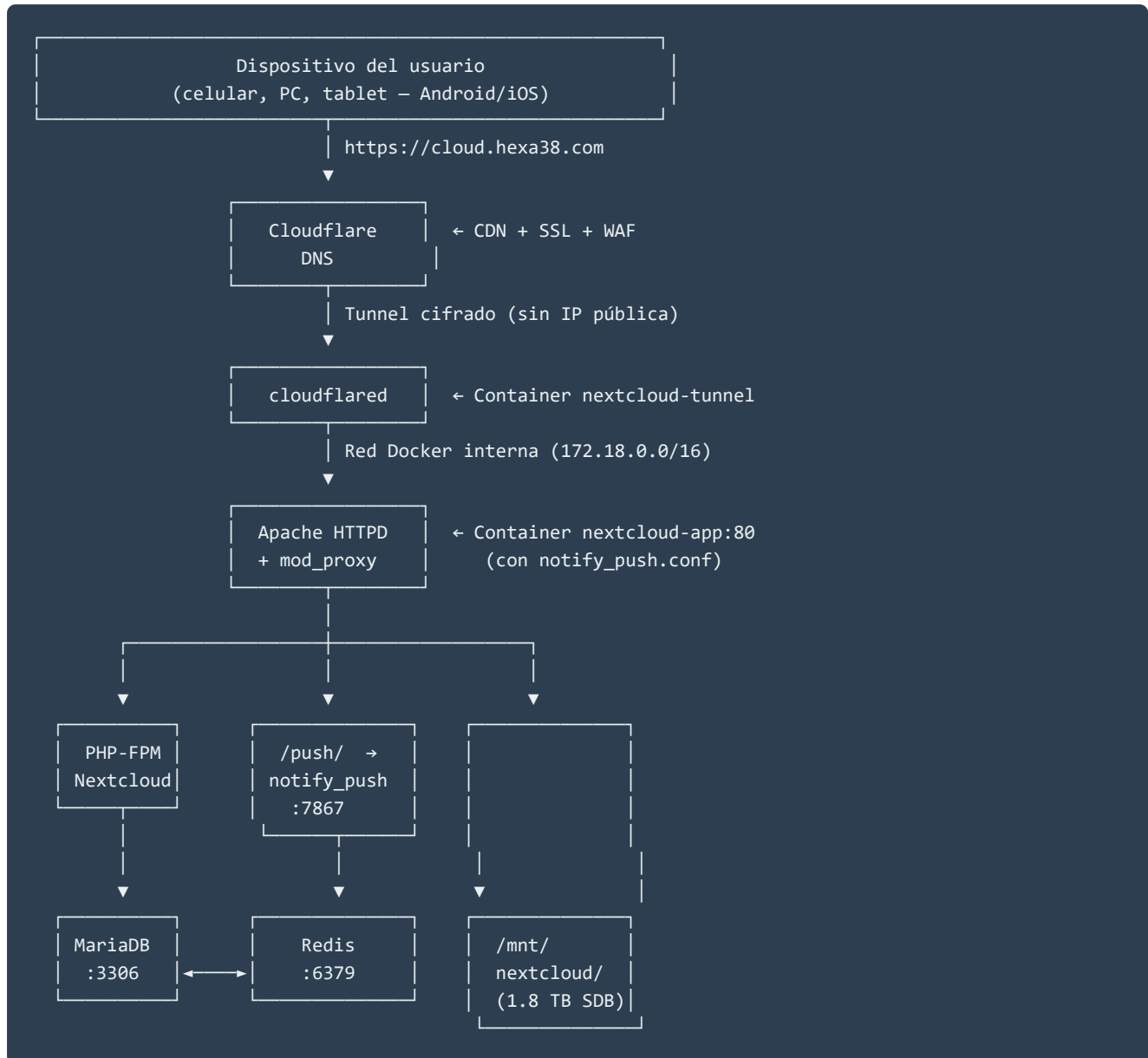
Disciplina	Aplicación
SysAdmin	Ubuntu Server, discos, swap, servicios
Redes	DNS, Cloudflare Tunnel, firewall, SSH, Tailscale
DevOps	Docker, Docker Compose, despliegue automatizado
Seguridad	HTTPS, 2FA, firewall, HSTS, hardening
Bases de datos	MariaDB, dumps, restauración, indexación
Programación/Scripting	Bash, PowerShell, automatización
Cloud Computing	Cloudflare, túneles, off-site backup
IaC	docker-compose.yml, .env, scripts versionables
Forense de sistemas	Análisis de logs, post-mortem, SMART
Reverse proxy	Apache mod_proxy, configuración de Cloudflare

4. Arquitectura del proyecto

4.1 Componentes físicos

Componente	Especificación	Función
NUC Intel N3150	4 GB RAM, WiFi/Ethernet	Servidor 24/7
Disco M.2 SSD	297 GB	Sistema operativo Ubuntu
Disco WD externo	2 TB (1.8 TB útiles)	Datos Nextcloud
Router	Conexión a internet	Conectividad

4.2 Diagrama de flujo de datos



4.3 Contenedores Docker

Contenedor	Imagen	Puerto	Función
<code>nextcloud-app</code>	<code>nextcloud:latest</code>	9000:80	App web + Apache reverse proxy
<code>nextcloud-db</code>	<code>mariadb:10.11</code>	3306 (interno)	Base de datos
<code>nextcloud-redis</code>	<code>redis:alpine</code>	6379 (interno)	Cache + file locking
<code>nextcloud-cron</code>	<code>nextcloud:latest</code>	-	Tareas programadas
<code>nextcloud-tunnel</code>	<code>cloudflare/cloudflared</code>	-	Túnel a Internet
<code>nextcloud-push</code>	<code>nextcloud:latest</code>	7867 (interno)	Notify push (sync rápido)

4.4 Estructura de archivos en el host

```

/home/aldo/nextcloud/
├─ docker-compose.yml      ← Definición de contenedores
├─ .env                    ← Variables sensibles (passwords, tokens)
├─ hsts.conf               ← Headers HSTS
├─ data-protect.conf       ← Protección /data
├─ notify_push.conf        ← Apache reverse proxy a notify_push
├─ backup.sh               ← Script de backup automático
└─ backup.sh.bak           ← Backup del script

/mnt/nextcloud/
├─ Disco SDB de 1.8 TB (ext4)
├─ Archivos PHP de Nextcloud
├─ html/
│   └─ config/
│       └─ config.php      ← Configuración principal
├─ data/                  ← Archivos de los usuarios
│   └─ Aldz/
│   └─ Alanav/
│   └─ Bianny Escobar/
│   └─ Kevin/
│   └─ Tiendas_PlayinVR/
├─ db/                   ← Archivos de MariaDB
└─ backups/              ← Backups automáticos
    └─ 2026-04-26_03-00/
        ├── database.sql
        ├── userdata/
        ├── config/
        ├── docker-compose.yml
        ├── .env
        └─ backup.log

```

Parte 2 — Implementación

5. Desarrollo paso a paso (Fases 1-7)

Fase 1: Configuración inicial en PC Windows (Prototipo)

- Instalación de Docker Desktop con WSL2
- Creación del `docker-compose.yml`
- Variables de entorno en `.env`
- Primer arranque: `docker compose up -d`
- Verificación: `curl http://localhost:9000/status.php`

Fase 2: Registro de dominio y acceso remoto

- Registro de **hexa38.com** en Cloudflare (\$10.11 USD/año)
- Configuración de Cloudflare Tunnel
- Subdominio `cloud.hexa38.com` → `nextcloud-app:80`
- HTTPS automático + HSTS

Fase 3: Optimización y seguridad

- `trusted_domains`: cloud.hexa38.com, localhost, IP local
- Redis como cache distribuido + file locking
- SMTP con Gmail
- 2FA/TOTP obligatorio
- Resolución de avisos de seguridad
- Migraciones MIME e índices BD

Fase 4: Sistema de backups automáticos

Cron diario a las 3 AM. El script:

1. Activa modo mantenimiento
2. Exporta BD con `mariadb-dump`
3. Copia datos de usuarios con `docker cp`
4. Copia configuración
5. Copia archivos Docker
6. Desactiva mantenimiento
7. Limpia backups antiguos (retención 7 días)

Fase 5: Migración a servidor dedicado (NUC)

- Instalación Ubuntu Server 24.04 LTS
- Formateo del disco WD 2 TB de NTFS a ext4
- Montaje permanente en `/mnt/nextcloud`
- Instalación Docker Engine
- Transferencia del backup vía SCP
- Restauración completa
- Redirección del Cloudflare Tunnel

Fase 6: Hardening del servidor

Medida	Herramienta	Descripción
Swap 4 GB	fallocate + mkswap	Memoria virtual
Firewall	UFW	Solo SSH y Nextcloud
Updates auto	unattended-upgrades	Parches de seguridad
Rotación logs	Docker daemon.json	Limita logs a 10 MB
Trusted proxies	config.php	IP real del usuario
Backup diario	cron + Bash	3 AM, 7 días

Fase 7: Resolución de avisos y SSH remoto

Avisos resueltos:

Aviso	Solución
Directorio /data accesible	data-protect.conf en Apache
Errores en logs	Limpieza histórica
2FA no obligatorio	Activación forzada
Migraciones MIME	<code>occ maintenance:repair --include-expensive</code>
Índices BD faltantes	<code>occ db:add-missing-indices</code>
HSTS no configurado	hsts.conf montado
Región telefónica	<code>default_phone_region=PE</code>
Mantenimiento	<code>maintenance_window_start=5</code> UTC

Tailscale para SSH remoto:

- VPN mesh basada en WireGuard

- IPs privadas 100.x.x.x
 - Acceso desde cualquier red sin abrir puertos
-

6. Archivos de configuración

6.1 docker-compose.yml (versión final actualizada)

```

services:
  # Base de datos
  db:
    image: mariadb:10.11
    container_name: nextcloud-db
    restart: always
    command: --transaction-isolation=READ-COMMITTED --log-bin=binlog --binlog-format=ROW
    volumes:
      - /mnt/nextcloud/db:/var/lib/mysql
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
    networks:
      - backend

  # Cache Redis
  redis:
    image: redis:alpine
    container_name: nextcloud-redis
    restart: always
    command: redis-server --requirepass ${REDIS_PASSWORD}
    networks:
      - backend

  # Aplicación Nextcloud
  app:
    image: nextcloud:latest
    container_name: nextcloud-app
    restart: always
    ports:
      - 9000:80
    depends_on:
      - db
      - redis
    volumes:
      - /mnt/nextcloud/html:/var/www/html
      - /mnt/nextcloud/data:/var/www/html/data
      - ./hsts.conf:/etc/apache2/conf-enabled/hsts.conf:ro
      - ./data-protect.conf:/etc/apache2/conf-enabled/data-protect.conf:ro
      - ./notify_push.conf:/etc/apache2/conf-enabled/notify_push.conf:ro
    environment:
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_HOST=db
      - REDIS_HOST=redis
      - REDIS_HOST_PASSWORD=${REDIS_PASSWORD}
      - NEXTCLOUD_TRUSTED_DOMAINS=${DOMAIN} localhost
      - OVERWRITEPROTOCOL=https
      - OVERWRITECLIURL=https://${DOMAIN}
      - OVERWRITEHOST=${DOMAIN}
      - PHP_MEMORY_LIMIT=1G

```

```

    - PHP_UPLOAD_LIMIT=16G
networks:
  - backend

# Cron
cron:
  image: nextcloud:latest
  container_name: nextcloud-cron
  restart: always
  volumes:
    - /mnt/nextcloud/html:/var/www/html
    - /mnt/nextcloud/data:/var/www/html/data
  entrypoint: /cron.sh
  depends_on:
    - app
  networks:
    - backend

# Cloudflare Tunnel
tunnel:
  image: cloudflare/cloudflared:latest
  container_name: nextcloud-tunnel
  restart: always
  command: tunnel run --token ${CLOUDFLARE_TOKEN}
  depends_on:
    - app
  networks:
    - backend

# Notify Push (sync instantáneo) – añadido en Fase 10
notify_push:
  image: nextcloud:latest
  container_name: nextcloud-push
  restart: always
  environment:
    - NEXTCLOUD_URL=http://app:80 # IMPORTANTE: interno, no público
  command: /var/www/html/custom_apps/notify_push/bin/x86_64/notify_push
/var/www/html/config/config.php
  volumes:
    - /mnt/nextcloud/html:/var/www/html:ro
  depends_on:
    - app
  networks:
    - backend

volumes:
  nextcloud_html:
  nextcloud_db:
  nextcloud_data:

networks:
  backend:
    driver: bridge

```

6.2 .env (variables sensibles)

```
DOMAIN=cloud.hexa38.com
MYSQL_ROOT_PASSWORD=*****
MYSQL_PASSWORD=*****
MYSQL_DATABASE=nextcloud
MYSQL_USER=nextcloud
REDIS_PASSWORD=*****
CLOUDFLARE_TOKEN=*****
```

6.3 hsts.conf

```
<IfModule mod_headers.c>
    Header always set Strict-Transport-Security "max-age=15552000; includeSubDomains"
</IfModule>
```

6.4 data-protect.conf

```
<Directory /var/www/html/data>
    Require all denied
    Options -Indexes
</Directory>
```

6.5 notify_push.conf (añadido v2)

```
# Cargar módulos de proxy si no están cargados
<IfModule !proxy_module>
    LoadModule proxy_module /usr/lib/apache2/modules/mod_proxy.so
</IfModule>
<IfModule !proxy_http_module>
    LoadModule proxy_http_module /usr/lib/apache2/modules/mod_proxy_http.so
</IfModule>
<IfModule !proxy_wstunnel_module>
    LoadModule proxy_wstunnel_module /usr/lib/apache2/modules/mod_proxy_wstunnel.so
</IfModule>

# Reverse proxy a notify_push
ProxyPass "/push/ws" "ws://nextcloud-push:7867/ws"
ProxyPassReverse "/push/ws" "ws://nextcloud-push:7867/ws"
ProxyPass "/push/" "http://nextcloud-push:7867/"
ProxyPassReverse "/push/" "http://nextcloud-push:7867/"
```

6.6 backup.sh (versión 2 con off-site)

```
#!/bin/bash
# Nextcloud Backup Script v2 - Local + Google Drive

set -e
DATE=$(date +%Y-%m-%d_%H-%M)
BACKUP_DIR="/mnt/nextcloud/backups/$DATE"
LOG_FILE="/mnt/nextcloud/backups/backup.log"
RETENTION_LOCAL=7
RETENTION_REMOTE=14
RCLONE_REMOTE="gdrive:nextcloud-backups"

export RCLONE_CONFIG=/home/aldo/.config/rclone/rclone.conf
set -a; source /home/aldo/nextcloud/.env; set +a

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

log "===== INICIO BACKUP ====="
mkdir -p "$BACKUP_DIR"

# 1. Modo mantenimiento ON
docker exec -u 0 nextcloud-app bash -c "php occ maintenance:mode --on"
trap 'docker exec -u 0 nextcloud-app bash -c "php occ maintenance:mode --off" || true; exit 1' ERR

# 2. Backup BD
docker exec nextcloud-db bash -c "mariadb-dump -u nextcloud -p\"$MYSQL_PASSWORD\" nextcloud --single-transaction" > "$BACKUP_DIR/database.sql"

# 3. Backup datos
docker cp nextcloud-app:/var/www/html/data "$BACKUP_DIR/userdata"

# 4. Backup config
docker cp nextcloud-app:/var/www/html/config "$BACKUP_DIR/config"

# 5. Archivos Docker
cp /home/aldo/nextcloud/docker-compose.yml "$BACKUP_DIR/"
cp /home/aldo/nextcloud/.env "$BACKUP_DIR/"
cp /home/aldo/nextcloud/hsts.conf "$BACKUP_DIR/"

# 6. Mantenimiento OFF
docker exec -u 0 nextcloud-app bash -c "php occ maintenance:mode --off"
trap - ERR

# 7. Limpiar locales antiguos
ls -dt /mnt/nextcloud/backups/20*/ | tail -n +$((RETENTION_LOCAL + 1)) | xargs -r rm -rf

# 8. Subir a Google Drive
if command -v rclone &> /dev/null; then
    TARFILE="/tmp/nextcloud_backup_$DATE.tar.gz"
    tar czf "$TARFILE" -C /mnt/nextcloud/backups "$DATE"
    rclone copy "$TARFILE" "$RCLONE_REMOTE/" --transfers=1 --retries=3
    rm -f "$TARFILE"

    # Limpiar remotos antiguos
    rclone lsf "$RCLONE_REMOTE/" --files-only | sort | head -n -"$RETENTION_REMOTE" | xargs -r -I{}
    rclone deletefile "$RCLONE_REMOTE/{}"
fi
```



```
log "===== BACKUP COMPLETADO ====="
```

6.7 Cron de root

```
# Ver con: sudo crontab -l
0 3 * * * /home/aldo/nextcloud/backup.sh
*/5 * * * * curl -fsS --retry 3 --max-time 30 https://cloud.hexa38.com/status.php > /dev/null 2>&1 &&
curl -fsS --max-time 10 https://hc-ping.com/TU_UUID > /dev/null 2>&1
```

6.8 Healthchecks en docker-compose.yml (añadido v3.1)

Cada servicio crítico tiene un **healthcheck** que monitorea su salud y permite a Docker reiniciarlo automáticamente si falla.

```
# db (MariaDB)
  healthcheck:
    test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
    interval: 30s
    timeout: 5s
    retries: 3
    start_period: 60s

# redis
  healthcheck:
    test: ["CMD-SHELL", "redis-cli -a \"$$REDIS_PASSWORD\" ping | grep -q PONG"]
    interval: 30s
    timeout: 5s
    retries: 3

# app (Apache+PHP)
  healthcheck:
    test: ["CMD-SHELL", "php -r 'exit(@file_get_contents(\"http://localhost/status.php\") ? 0 : 1);'"]
    interval: 60s
    timeout: 10s
    retries: 3
    start_period: 120s

# notify_push
  healthcheck:
    test: ["CMD-SHELL", "pgrep -f notify_push > /dev/null"]
    interval: 30s
    timeout: 5s
    retries: 3
    start_period: 30s
```

Verificar:

```
docker ps --format "table {{.Names}}\t{{.Status}}"
# Esperado: cada uno con estado "(healthy)"
```

7. Infrastructure as Code

7.1 Definición

Gestionar infraestructura mediante código versionable, en lugar de configuraciones manuales.

7.2 Beneficios

- **Reproducibilidad:** misma config = mismo resultado
- **Versionado:** Git registra cambios
- **Documentación automática:** el código es la documentación
- **Escalabilidad:** replicar es copiar archivos

7.3 Cómo reproducir el proyecto

```
# 1. Instalar Docker
curl -fsSL https://get.docker.com | sudo sh

# 2. Copiar archivos de configuración
scp -r usuario@servidor-origen:~/nextcloud ~/nextcloud

# 3. Levantar infraestructura
cd ~/nextcloud && docker compose up -d

# 4. Restaurar backup
docker exec -i nextcloud-db bash -c \
'mariadb -u root -p"$MYSQL_ROOT_PASSWORD" nextcloud' < backup/database.sql
```

8. Costos del proyecto

Concepto	Costo	Frecuencia
Dominio hexa38.com	\$10.11 USD	Anual
Cloudflare Tunnel	Gratis	-
Nextcloud	Gratis	-
Docker Engine	Gratis	-
Ubuntu Server	Gratis	-
NUC (hardware)	Ya existente	-
Disco WD 2 TB	Ya existente	-
Electricidad (~15W)	~\$2-3 USD	Mensual
Google Drive (backup off-site)	Gratis (15 GB)	-
Tailscale	Gratis	-
TOTAL ANUAL	~\$46 USD	

Ahorro vs servicios comerciales: 60-68%

Parte 3 — Evolución y Mejoras Aplicadas

9. Fase 8: Incidente y recuperación de datos

9.1 ¿Qué pasó?

Síntomas:

- Nextcloud aparecía como instalación nueva
- Solo 1 usuario admin visible (5 usuarios desaparecieron)
- 20 errores en logs
- Avisos de "Modo mantenimiento" en clientes

Causa raíz identificada:

1. **Pila CMOS agotada** → BIOS reseteada → múltiples encendidos/apagados
2. **Apagado forzado** durante operaciones de Docker
3. **Filesystem ext4 quedó "dirty"** tras apagado forzado
4. **Recreación de contenedor** durante el problema
5. **Bind mount** apareció vacío al backend después del reboot

9.2 Diagnóstico aplicado (paso a paso)

```
# 1. Estado de contenedores
docker ps
docker ps -a

# 2. Estado de discos y montajes
mount | grep nextcloud
lsblk -f
df -h /mnt/nextcloud
sudo du -sh /mnt/nextcloud/*

# 3. Verificar volúmenes Docker
docker volume ls
docker inspect nextcloud-app --format '{{range .Mounts}}{{.Type}} {{.Source}} -> {{.Destination}}
{{println}}{{end}}'

# 4. Buscar backups
ls -la /mnt/nextcloud/backups/
sudo find / -name "*.sql" 2>/dev/null | grep -v snap

# 5. Forzar journal replay del filesystem
cd /home/aldo/nextcloud && docker compose stop
sudo umount /mnt/nextcloud
sudo mount /mnt/nextcloud
docker compose start
```

9.3 Procedimiento de restauración

```
# Variables
export BACKUP=/mnt/nextcloud/backups/2026-04-24_03-00
set -a; source /home/aldo/nextcloud/.env; set +a

# Paso 1: Modo mantenimiento + parar contenedores que tocan datos
docker exec -u www-data nextcloud-app php occ maintenance:mode --on
cd /home/aldo/nextcloud
docker compose stop app cron tunnel notify_push

# Paso 2: Restaurar BD
docker exec -i nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" -e \
  "DROP DATABASE IF EXISTS nextcloud; CREATE DATABASE nextcloud CHARACTER SET utf8mb4 COLLATE
  utf8mb4_bin;"
docker exec -i nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" nextcloud < $BACKUP/database.sql

# Verificar usuarios
docker exec nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" nextcloud -e "SELECT uid FROM
oc_users;"

# Paso 3: Restaurar userdata
sudo rm -rf /mnt/nextcloud/data/* /mnt/nextcloud/data/.[!.*]*
sudo cp -a $BACKUP/userdata/. /mnt/nextcloud/data/
sudo chown -R www-data:www-data /mnt/nextcloud/data

# Paso 4: Restaurar config
sudo cp -a $BACKUP/config/. /mnt/nextcloud/html/config/
sudo chown -R www-data:www-data /mnt/nextcloud/html/config

# Paso 5: Reiniciar y validar
docker compose start
sleep 15
docker exec -u www-data nextcloud-app php occ maintenance:mode --off
docker exec -u www-data nextcloud-app php occ user:list
docker exec -u www-data nextcloud-app php occ files:scan --all
```

9.4 Lecciones aprendidas

✗ Evitar	✓ Hacer
Apagado forzado	<code>sudo shutdown -h now</code>
<code>docker compose down -v</code>	<code>docker compose down</code> (sin -v)
Editar compose sin backup	<code>bash backup.sh</code> antes de cambios
Tener Snap Nextcloud	<code>sudo snap remove --purge nextcloud</code>
Pila CMOS sin cambiar	Cambiar cada 5-10 años

10. Fase 9: Backup off-site (Google Drive)

10.1 Por qué off-site

El backup local protege contra:

-  Borrado accidental
-  Corrupción del filesystem

Pero NO protege contra:

-  Falla total del disco SDB
-  Robo / incendio del NUC
-  Ransomware que cifre todo el disco

Solución: regla 3-2-1

- **3** copias de los datos
- **2** medios diferentes
- **1** copia off-site

10.2 Configuración de rclone con Google Drive

```
# 1. Instalar rclone
sudo apt install -y rclone

# 2. Configurar Google Drive (modo headless)
rclone config

# Respuestas:
#   n (new remote)
#   name: gdrive
#   storage: drive
#   client_id: (vacío)
#   client_secret: (vacío)
#   scope: drive
#   service_account_file: (vacío)
#   advanced config: n
#   auto config: n   ← IMPORTANTE: headless

# Te dará un comando como:
#   rclone authorize "drive" "eyJzY29wZSI6...."

# 3. En Windows con navegador (PowerShell):
winget install Rclone.Rclone
rclone authorize "drive" "eyJzY29wZSI6...."
# Autoriza en navegador, copia el token JSON

# 4. Pega el token en el NUC en config_token>
# 5. Responde: n, y, q

# 6. Verificar
rclone lsd gdrive:
rclone mkdir gdrive:nextcloud-backups
```

10.3 Modificar backup.sh

Añadir al script:

```
export RCLONE_CONFIG=/home/aldo/.config/rclone/rclone.conf
```

Y al final:

```
TARFILE="/tmp/nextcloud_backup_$(date +%Y%m%d).tar.gz"
tar czf "$TARFILE" -C /mnt/nextcloud/backups "$(date +%Y%m%d)"
rclone copy "$TARFILE" "gdrive:nextcloud-backups/" --transfers=1 --retries=3
rm -f "$TARFILE"

# Retención remota: 14 días
rclone lsf "gdrive:nextcloud-backups/" --files-only | sort | head -n -14 | xargs -r -I{} rclone
deletefile "gdrive:nextcloud-backups/{"
```

10.4 Verificación

```
sudo bash /home/aldo/nextcloud/backup.sh
rclone ls gdrive:nextcloud-backups/
rclone size gdrive:nextcloud-backups/
```

11. Fase 10: Notify Push (sincronización instantánea)

11.1 Qué hace

Permite que los clientes desktop/móvil reciban cambios **en tiempo real** vía WebSocket, en vez de hacer polling cada 30 segundos.

11.2 Arquitectura

```

Cliente desktop —ws→ Cloudflare —ws→ cloudflared
                        |
                        ▼
                  Apache (nextcloud-app)
                        |
                  /push/* → ProxyPass
                        |
                        ▼
                  nextcloud-push:7867
                        |
                        ▼
                  Redis (notificaciones)
  
```

11.3 Pasos de configuración (resumen)

Paso 1: Añadir contenedor `notify_push` a `docker-compose.yml`

```
notify_push:
  image: nextcloud:latest
  container_name: nextcloud-push
  restart: always
  environment:
    - NEXTCLOUD_URL=http://app:80    # ← Crítico: interno, no público
  command: /var/www/html/custom_apps/notify_push/bin/x86_64/notify_push
/var/www/html/config/config.php
  volumes:
    - /mnt/nextcloud/html:/var/www/html:ro
  depends_on:
    - app
  networks:
    - backend
```

Paso 2: Crear `notify_push.conf` (Apache reverse proxy)

```
cat > /home/aldo/nextcloud/notify_push.conf << 'EOF'
<IfModule !proxy_module>
    LoadModule proxy_module /usr/lib/apache2/modules/mod_proxy.so
</IfModule>
<IfModule !proxy_http_module>
    LoadModule proxy_http_module /usr/lib/apache2/modules/mod_proxy_http.so
</IfModule>
<IfModule !proxy_wstunnel_module>
    LoadModule proxy_wstunnel_module /usr/lib/apache2/modules/mod_proxy_wstunnel.so
</IfModule>

ProxyPass "/push/ws" "ws://nextcloud-push:7867/ws"
ProxyPassReverse "/push/ws" "ws://nextcloud-push:7867/ws"
ProxyPass "/push/" "http://nextcloud-push:7867/"
ProxyPassReverse "/push/" "http://nextcloud-push:7867/"
EOF
```

Paso 3: Montar config en `docker-compose.yml` (servicio app)

```
volumes:
  - ./notify_push.conf:/etc/apache2/conf-enabled/notify_push.conf:ro
```

Paso 4: Recrear contenedores

```
cd /home/aldo/nextcloud
docker compose up -d --force-recreate app notify_push
```


Paso 5: Configurar trusted_proxies y forwarded_for_headers

```
docker exec -u www-data nextcloud-app php occ config:system:set forwarded_for_headers 0 --
value="HTTP_X_FORWARDED_FOR"
```

Paso 6: Setup final

```
set -a; source /home/aldo/nextcloud/.env; set +a
docker exec -u www-data nextcloud-app php occ notify_push:setup https://${DOMAIN}/push
```

✓ Salida esperada:

```
✓ redis is configured
✓ push server is receiving redis messages
✓ push server can load mount info from database
✓ push server can connect to the Nextcloud server
✓ push server is a trusted proxy
✓ push server is running the same version as the app
configuration saved
```

11.4 Por qué `NEXTCLOUD_URL=http://app:80` y no el dominio público

Si pones `NEXTCLOUD_URL=https://${DOMAIN}` :

- notify_push hace requests al dominio público
- Cloudflare añade tu IP pública al X-Forwarded-For
- Esa IP no está en trusted_proxies
- Falla la validación de proxy confiable

Con `NEXTCLOUD_URL=http://app:80` :

- notify_push usa la red Docker interna (172.18.0.0/16)
- Esa red Sí está en trusted_proxies
- Todo funciona limpiamente
- **Los clientes externos siguen usando <https://cloud.hexa38.com/push>** (lo que configuraste con `notify_push:setup`)

12. Fase 11: Diagnóstico forense del sistema

12.1 Por qué se hizo

Tras el incidente, se realizó un análisis profundo del hardware y software para descartar problemas físicos y entender qué causó el incidente.

12.2 Hallazgos

Hardware:

-  **CPU N3150:** temperatura normal (52-57°C)
-  **RAM 3.68 GB:** sin errores EDAC/MCE, justa pero suficiente
-  **Disco SDB (datos 1.8 TB):** SMART PASSED, todos los contadores en 0
-  **Disco SDA (sistema):** HDD con 11 sectores reasignados, envejecido pero funcional
-  **Sin kernel panics, sin OOM**

Software:

-  Reboots cortos explicados: usuario configurando BIOS tras cambio de pila CMOS
-  Apagados limpios (no crashes)
-  Snap Nextcloud detectado (a eliminar)

12.3 Comandos de diagnóstico clave

```
# Hardware
sudo smartctl -H /dev/sda          # Salud rápida
sudo smartctl -A /dev/sda          # Atributos detallados
sudo sensors                       # Temperaturas
free -h                           # RAM
sudo dmesg | grep -i error          # Errores kernel

# Sistema
uptime                             # Tiempo encendido
sudo last reboot                    # Historial de reboots
sudo journalctl --list-boots        # Lista de boots
sudo journalctl -b -1 | tail -50    # Logs del boot anterior
systemctl --failed                  # Servicios caídos

# Forense
sudo journalctl --since "2 days ago" | grep -iE "panic|oops|killed"
sudo journalctl -k -b 0 | grep -iE "memory|edac|mce"
cat /home/aldo/.bash_history | tail -100
```

13. Fase 12: Sistema de monitoreo en capas

13.1 Filosofía: defense in depth

Tres capas independientes que se complementan:

CAPA 1: Auto-recuperación (Docker healthchecks)
Inmediato (silencioso) – Reinicia procesos colgados

CAPA 2: Monitoreo de aplicación (Healthchecks.io)
10-15 min – Avisa si Nextcloud no responde

CAPA 3: Monitoreo de infraestructura (Cloudflare)
1-2 min ⚡ – Avisa si tunnel/NUC pierde conectividad

13.2 Capa 1 — Healthchecks de Docker

Cada contenedor crítico tiene su healthcheck nativo. Si el healthcheck falla N veces consecutivas, el contenedor se marca **unhealthy** y Docker lo reinicia (gracias a **restart: always**).

Healthchecks añadidos al `docker-compose.yml` :

nextcloud-db (MariaDB)

```
healthcheck:
  test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 60s
```

Usa el script oficial de la imagen mariadb:10.11.

nextcloud-redis

```
healthcheck:
  test: ["CMD-SHELL", "redis-cli -a \"$$REDIS_PASSWORD\" ping | grep -q PONG"]
  interval: 30s
  timeout: 5s
  retries: 3
  environment:
    - REDIS_PASSWORD=${REDIS_PASSWORD}
```

Verifica que Redis responde a **PING** con **PONG**.

nextcloud-app (Apache + PHP)

```
healthcheck:
  test: ["CMD-SHELL", "php -r 'exit(@file_get_contents(\"http://localhost/status.php\") ? 0 : 1);'"]
  interval: 60s
  timeout: 10s
  retries: 3
  start_period: 120s
```

PHP en lugar de curl (la imagen Nextcloud no trae curl). Verifica que `/status.php` responde.

nextcloud-push (notify_push)

```
healthcheck:
  test: ["CMD-SHELL", "pgrep -f notify_push > /dev/null"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 30s
```

Verifica que el proceso `notify_push` esté vivo dentro del contenedor.

Sin healthcheck: cron y tunnel

- `nextcloud-cron` : ejecuta tareas, no expone endpoint
- `nextcloud-tunnel` : ya monitoreado por Cloudflare (Capa 3)

Verificar healthchecks:

```
docker ps --format "table {{.Names}}\t{{.Status}}"
# Esperado: cada uno con (healthy)

docker inspect nextcloud-app --format '{{json .State.Health}}' | python3 -m json.tool
```

13.3 Capa 2 — Healthchecks.io (servicio externo gratuito)

Servicio "dead-man-switch": si **deja de recibir** un ping, te avisa.

Por qué es genial:

- ☒ Gratis hasta 20 monitores
- ☒ Detecta si tu casa pierde Internet (el NUC no puede enviar ping)
- ☒ Detecta si el NUC está apagado
- ☒ Detecta si Cloudflare cae
- ☒ Detecta si Nextcloud devuelve error
- ☒ Notifica por email, Telegram, Discord, Slack, Webhook

Setup paso a paso

1. Crear cuenta en <https://healthchecks.io> (gratis)

2. Crear un check llamado `Nextcloud Hexa38` :

- Schedule: Simple
- Period: 10 minutes
- Grace Time: 5 minutes
- Slug: nextcloud-hexa38 (opcional)

3. Copiar la Ping URL (algo como `https://hc-ping.com/UUID`)

4. Probar manualmente desde el NUC:

```
curl -fsS https://hc-ping.com/TU_UUID
# Debe responder: OK
```

5. Añadir al cron de root (sin abrir editor):

```
(sudo crontab -l 2>/dev/null; echo "*/5 * * * * curl -fsS --retry 3 --max-time 30
https://cloud.hexa38.com/status.php > /dev/null 2>&1 && curl -fsS --max-time 10 https://hc-
ping.com/TU_UUID > /dev/null 2>&1") | sudo crontab -
```

Verificar:

```
sudo crontab -l
```

Debe mostrar **2 líneas** (backup + monitoreo).

Cómo funciona la lógica

```
Cada 5 minutos cron:
  curl https://cloud.hexa38.com/status.php
    |
    |— responde OK → curl https://hc-ping.com/UUID (silencioso)
    |
    |— falla → NO hace ping a healthchecks.io
```

Healthchecks.io espera cada 10 min un ping.
Si pasan 15 min (10 + 5 grace) sin ping → ALERTA POR EMAIL

Configurar canales de notificación

En healthchecks.io → **Integrations** → activa los que prefieras:

-  Email (default, gratis)
-  Telegram (recomendado, instantáneo y gratis)
-  Discord / Slack (gratis)
-  Webhook (gratis, cualquier URL)
-  WhatsApp / SMS / llamada (REQUIERE plan de pago Hobbyist ~\$5/mes — redundante si ya tienes Telegram)

Setup específico de Telegram (gratis, recomendado)

Healthchecks.io tiene un bot oficial llamado `@HealthchecksBot`. Setup en 3 minutos:

1. **En healthchecks.io:** tu check → Integrations → busca **Telegram** → click **Add Integration**
2. **En Telegram:** busca `@HealthchecksBot` → envía `/start`
3. El bot te responde con un **link de confirmación** tipo:

```
https://healthchecks.io/integrations/add_telegram/?eyJpZCI6...
```

4. **Click en el link** → te abre healthchecks.io
5. Selecciona el proyecto → click **Connect Telegram**
6. **Activa el toggle de Telegram** en tu check

Probar la notificación

En healthchecks.io → tu check → botón "**Send a Test Notification**". Debes recibir un mensaje en Telegram instantáneamente.

Por qué NO WhatsApp

Aunque healthchecks.io ofrece WhatsApp, lo desestimamos porque:

- Requiere plan pago Hobbyist (~\$5/mes = ~\$60/año)
- Cada mensaje vía Twilio cuesta dinero (\$0.005-0.05 USD)
- Telegram da el **mismo efecto instantáneo gratis**
- Mantener costos bajos es uno de los objetivos del proyecto

13.4 Capa 3 — Cloudflare Tunnel Health Alerts

Cloudflare detecta caídas del tunnel **antes** que healthchecks.io (1-2 min vs 10-15 min). Setup:

1. Ir a <https://one.dash.cloudflare.com/>
2. **Notifications** (en el dashboard general, no en el tunnel)
3. **Add** → buscar "**Túnel**" → seleccionar "**Alerta de estado del túnel**"
4. **Configurar:**
 - Name: `Hexa38 Tunnel Down`
 - Tunnels: seleccionar tu tunnel (en este proyecto: `servidor-cas`)
 - Activador: "**O se degrada o se reduce**" (cubre degradación parcial y caída total)
 - Email: tu correo
5. **Crear** y confirmar el email de suscripción
6. **Test** (opcional):

```
docker stop nextcloud-tunnel
# Esperar 1-2 minutos → debe llegar email "El túnel ahora está en estado caído"
docker start nextcloud-tunnel
```

13.5 Tabla de cobertura de monitoreo

Escenario	Detector	Canal	Tiempo
Apache/PHP se cuelga	Docker healthcheck → restart	(silencioso)	Inmediato
MariaDB no responde	Docker healthcheck → restart	(silencioso)	Inmediato
Redis cae	Docker healthcheck → restart	(silencioso)	Inmediato
notify_push muere	Docker healthcheck → restart	(silencioso)	Inmediato
Nextcloud devuelve 500	Healthchecks.io	 Telegram +  Email	10-15 min
BD inaccesible (config rota)	Healthchecks.io	 Telegram +  Email	10-15 min
Tunnel pierde conectividad	Cloudflare	 Email	1-2 min ⚡
NUC apagado	Cloudflare + Healthchecks.io	 Email +  Telegram	1-2 min ⚡
Internet en casa cae	Cloudflare	 Email	1-2 min ⚡
Disco lleno	(manual: revisar logs)	-	-

13.6 Mantenimiento del sistema de monitoreo

Mensual:

- Revisar dashboard de healthchecks.io (debería estar siempre verde)
- Revisar emails de Cloudflare (cualquier alerta)
- Ejecutar `docker ps` y confirmar que todos están `(healthy)`

Semestral:

- Probar el sistema apagando el tunnel a propósito (debe llegar alerta)
- Verificar que la URL de healthchecks.io sigue activa
- Renovar canales de notificación si cambiaste de email/Telegram

13.7 Cuando escalar a más monitoreo

Si tu uso crece (>20 usuarios, >5 servicios, presupuesto pago), considera:

Herramienta	Cuándo
Uptime Kuma	Si quieres dashboard self-hosted con histórico
Netdata	Si quieres métricas detalladas (CPU, RAM, disk I/O)
Prometheus + Grafana	Producción seria con SLAs
Sentry	Si necesitas tracking de errores PHP/JS

Para uso personal/familiar, las 3 capas actuales son **más que suficientes**.

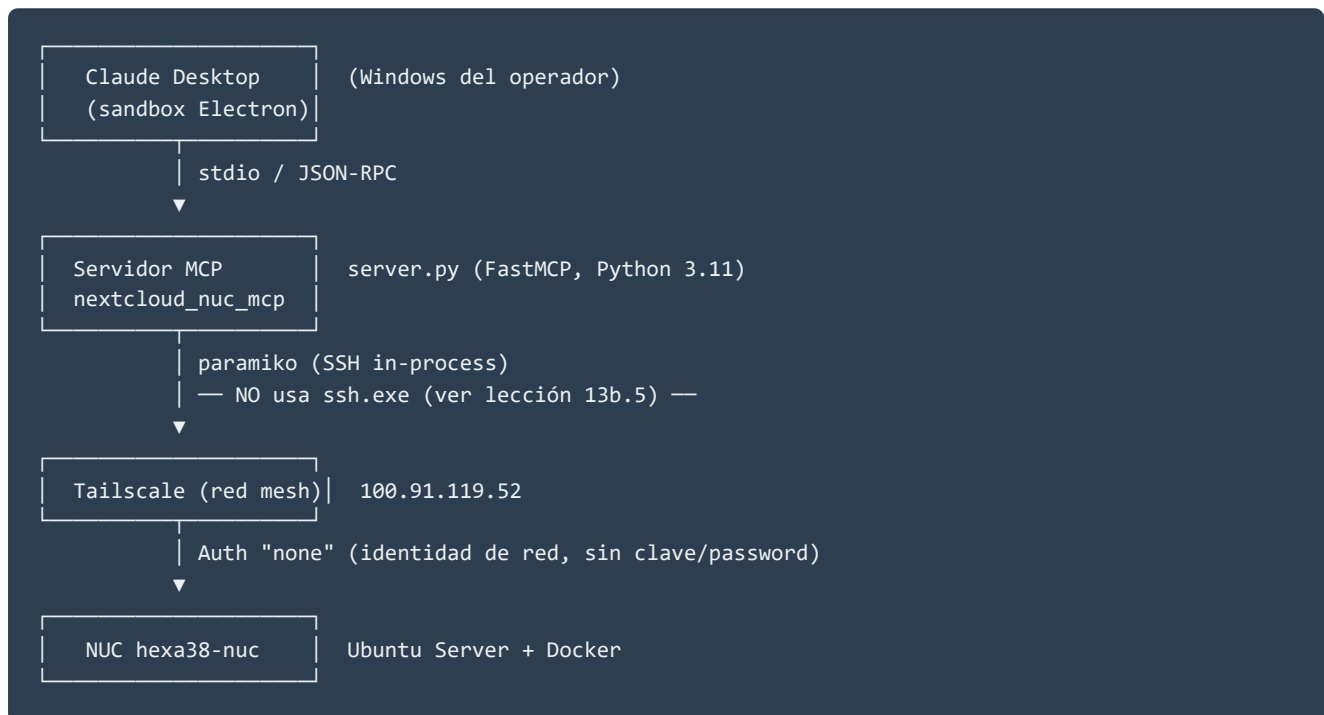
13b. Fase 13: MCP de gestión del NUC (Claude)

13b.1 Qué es y por qué

Un **servidor MCP (Model Context Protocol)** permite que Claude (la IA) diagnostique y opere el NUC directamente desde una conversación, sin que el operador tenga que abrir SSH y ejecutar comandos manualmente. Es la evolución natural del sistema de monitoreo: el monitoreo **avisa** del problema; el MCP permite **diagnosticarlo y resolverlo** de forma asistida.

Flujo: el usuario escribe "revisa el túnel" → Claude invoca la herramienta MCP → el servidor MCP se conecta por SSH (Tailscale) al NUC, ejecuta el comando, devuelve el resultado ya analizado.

13b.2 Arquitectura



13b.3 Herramientas expuestas

Herramienta	Función	Tipo
<code>nuc_check_health</code>	Diagnóstico completo (RAM, disco, contenedores, errores, backup)	read-only
<code>nuc_get_containers</code>	Estado de los 6 contenedores con healthcheck	read-only
<code>nuc_get_logs</code>	Logs de un contenedor (filtro por líneas o tiempo)	read-only
<code>nuc_restart_service</code>	Reinicia un contenedor específico	acción
<code>nuc_run_occ</code>	Ejecuta comandos <code>occ</code> de Nextcloud	acción
<code>nuc_check_backup</code>	Estado de backups locales y Google Drive	read-only
<code>nuc_check_disk</code>	Uso de disco + SMART de sda/sdb	read-only
<code>nuc_check_tunnel</code>	Diagnóstico del túnel Cloudflare (estado, logs, reconexiones)	read-only
<code>nuc_maintenance_mode</code>	Activa/desactiva modo mantenimiento	acción
<code>nuc_run_command</code>	Comando shell arbitrario (diagnóstico avanzado)	acción
<code>nuc_debug_connection</code>	Verifica la conectividad MCP → NUC	read-only

Cada herramienta declara `annotations` (`readOnlyHint`, `destructiveHint`, etc.) para que Claude sepa cuáles son seguras y cuáles requieren confirmación.

13b.4 Estructura de archivos del MCP

```
mcp-nextcloud/
├─ server.py      ← servidor FastMCP con las 11 herramientas
├─ requirements.txt ← mcp[cli], paramiko, python-dotenv
└─ .env          ← NUC_HOST, NUC_USER, puerto, rutas (sin secretos: Tailscale auth none)
```

Registro en `%APPDATA%\Claude\claude_desktop_config.json`:

```
{
  "mcpServers": {
    "nextcloud-nuc": {
      "command": "C:\\...\\python.exe",
      "args": ["C:\\...\\mcp-nextcloud\\server.py"]
    }
  }
}
```

13b.5 Lección clave: por qué paramiko y no `ssh.exe`

Síntoma: al lanzar las herramientas desde Claude, todo fallaba con `exit 255` y `stderr completamente vacío`, incluso con `ssh -vvv` redirigido a un archivo. Pero `ssh.exe` funcionaba perfecto desde una terminal PowerShell normal.

Diagnóstico (metodología):

1. Test TCP al puerto 22 desde el proceso MCP → **OK** (la red llega)
2. Test `cmd /c echo` como subprocesso desde el MCP → **OK** (subprocess funciona)
3. Test `ssh.exe -vvv` con stderr a archivo → **exit 255, cero bytes** (ni el banner de versión)

Causa raíz: Claude Desktop ejecuta el MCP en un **sandbox de Electron** que bloquea específicamente la ejecución de `ssh.exe` como proceso hijo. No es problema de red, ni de pipes, ni de credenciales: es ese binario concreto.

Solución: usar **paramiko** (cliente SSH puro en Python), que corre **dentro** del proceso Python y no lanza binarios externos. Como el proceso Python sí tiene acceso de red (lo probé el test TCP), paramiko conecta sin problema.

Detalle de Tailscale SSH: el NUC usa Tailscale SSH, que autentica por **identidad de red** (`auth_none` en paramiko), sin clave ni contraseña. El handshake muestra `Authentication (none) successful!`.

```
# Núcleo del helper SSH (paramiko, auth none de Tailscale)
sock = socket.create_connection((NUC_HOST, 22), timeout=15)
transport = paramiko.Transport(sock)
transport.start_client(timeout=15)
transport.auth_none(NUC_USER)          # Tailscale SSH: identidad de red
chan = transport.open_session()
chan.exec_command(command)
```

El helper incluye **reintento automático** (1 retry con backoff) porque el WiFi intermitente del NUC puede causar fallos transitorios de conexión.

13b.6 Buenas prácticas aplicadas (metodología MCP)

- **Lista blanca de contenedores:** las herramientas sólo aceptan los 6 contenedores conocidos (evita inyección de nombres arbitrarios).
- **Validación con Pydantic:** cada entrada se valida con modelos (`extra="forbid"` , rangos en límites).
- **Annotations correctas:** read-only vs destructivo, para que Claude pida confirmación sólo cuando toca.
- **Logging de paramiko silenciado** a nivel WARNING (evita ruido en el canal stdio del MCP).
- **Sin secretos en el `.env`:** Tailscale autentica por identidad; no hay claves ni passwords en texto plano.
- **`sudo` sigue pidiendo contraseña:** el MCP **no** puede ejecutar comandos privilegiados de forma autónoma (decisión de seguridad). Los cambios root los aplica el operador manualmente.

13c. Fase 14: Inestabilidad WiFi y degradación del túnel

13c.1 El síntoma

Cloudflare enviaba correos recurrentes de **"túnel degradado"**. Habían empezado tras una caída del servicio de internet que provocó un **cambio automático en los nombres de las interfaces de red**. La

reconexión se había resuelto, pero las alertas continuaban.

13c.2 Diagnóstico (de lo general a lo específico)

Paso 1 — Logs del túnel (`nuc_check_tunnel`): el contenedor `nextcloud-tunnel` estaba `running` con `Restarts: 0`, pero los logs mostraban tormentas de:

```
ERR failed to accept incoming stream requests error="failed to accept QUIC stream:
    timeout: no recent network activity" connIndex=0..3
INF Retrying connection in up to 1s
INF Registered tunnel connection ...
```

Las **4 conexiones QUIC** a Cloudflare caían **simultáneamente** y se reconectaban. Contador de eventos de reconexión: **12 046 en ~4 días**. Cuando las 4 caen a la vez, el problema no es de Cloudflare sino de la **conectividad local**.

Paso 2 — Estado de las interfaces (`ip -brief addr`):

```
enp3s0    DOWN                                ← Ethernet: SIN cable
wlp2s0    UP      192.168.1.69/24            ← WiFi: lleva todo el tráfico
```

La ruta por defecto salía por **WiFi** (`default via 192.168.1.1 dev wlp2s0`). El Ethernet quedó caído tras el incidente.

Paso 3 — Logs de red del sistema (`journalctl`): el momento exacto de la degradación coincidía con el WiFi:

```
22:22:59 wlp2s0: Lost carrier           ← WiFi pierde señal
22:23:04 (túnel) las 4 conexiones QUIC caen ← 5 s después
22:23:10 wlp2s0: Gained carrier         ← WiFi vuelve
```

Tormenta de `Lost carrier` / `Gained carrier` / `DHCP lease lost` repetidos.

Paso 4 — Descartar señal débil (`/proc/net/wireless`, `lspci`): la señal era **fuerte** (-49 dBm, calidad 61/70). El chip: **Intel Wireless 3160**, driver `iwlwifi`. Señal buena + cortes de carrier = no es problema físico, es el **bug de gestión de energía del driver** `iwlwifi`.

13c.3 Causa raíz

El driver `iwlwifi` del Intel 3160 aplica un **ahorro de energía agresivo** que apaga momentáneamente la radio aunque la señal sea excelente. Cada micro-corte mata las 4 conexiones QUIC del túnel → Cloudflare lo detecta como degradado → envía la alerta. Luego el WiFi se recupera y el túnel se reconecta. **Las alertas eran reales y correctas**; el túnel estaba bien configurado.

13c.4 La solución aplicada

Desactivar la gestión de energía del WiFi de forma **persistente** (sobrevive reinicios):

```
# Crear configuración persistente del módulo
sudo tee /etc/modprobe.d/iwlwifi.conf > /dev/null << 'EOF'
# Estabilidad Intel Wireless 3160 – evita cortes de carrier
options iwlwifi power_save=0 d0i3_disable=1 uapsd_disable=1
options iwlvm power_scheme=1
EOF

# Instalar 'iw' (diagnóstico de WiFi)
sudo apt install -y iw

# Aplicar (recargar módulo o reiniciar el NUC)
sudo modprobe -r iwlvm iwlwifi && sudo modprobe iwlwifi

# Verificar
iw dev wlp2s0 get power_save      # → Power save: off
```

Opción	Efecto
<code>power_save=0</code>	Desactiva el ahorro de energía del WiFi (causa principal)
<code>d0i3_disable=1</code>	Desactiva un estado de bajo consumo problemático del 3160
<code>uapsd_disable=1</code>	Desactiva el ahorro automático por inactividad
<code>power_scheme=1</code> (iwlvm)	Modo de rendimiento "activo" en el firmware

Resultado verificado: `Power save: off`, configuración persistente en su sitio.

Parámetros del módulo efectivamente activos (`/sys/module/iwlwifi/parameters/`), verificados vía MCP:

Parámetro	Valor activo	Estado
<code>power_save</code>	<code>N</code> (off)	✅ aplicado
<code>uapsd_disable</code>	<code>1</code>	✅ aplicado
<code>iwlvm power_scheme</code>	<code>1</code>	✅ aplicado
<code>d0i3_disable</code>	(vacío)	🚫 no soportado en esta versión del driver — inofensivo

El reload del módulo (`modprobe -r/modprobe`) aplicó los parámetros sin necesidad de reiniciar (uptime se mantuvo en 4 días al verificar).

13c.4-bis Evidencia de mejora — antes/después (verificado con MCP)

Todas las cifras provienen del journal del kernel del NUC, consultadas con la herramienta `nuc_run_command` del MCP (`journalctl ... | grep -ciE 'Lost carrier|Gained carrier'`). El **fix se aplicó el 2026-06-01 05:40 UTC** (marca de tiempo de `/etc/modprobe.d/iwlwifi.conf`).

Métrica	ANTES (sin fix)	DESPUÉS (con fix)
Ventana de referencia	2026-05-30 22:00–23:00	2026-06-01 07:00 → 06-02 00:30 (~17.5 h limpias)
Eventos de carrier WiFi/hora (promedio)	52 / hora (tormenta)	~0.7 / hora
Horas con CERO eventos	0 de 1	16 de 17
Desconexiones del túnel ("no recent network activity")	continuas (cada pocos min)	0 en 18.5 h
Patrón en logs del túnel	caídas de las 4 conexiones QUIC cada pocos minutos	sin caídas; un parpadeo WiFi aislado que el túnel resistió

Reducción de inestabilidad: ~98.7 % (52 → 0.7 eventos/hora). Y, lo más relevante para el síntoma original: **el túnel dejó de caerse** — cero alertas de degradación atribuibles a la red durante la ventana medida.

Metodología de medición (reproducible):

```
# ANTES (ventana de tormenta conocida)
journalctl --since '2026-05-30 22:00' --until '2026-05-30 23:00' \
| grep -ciE 'Lost carrier|Gained carrier'      # → 52

# DESPUÉS (ventana post-fix, ajustar fechas)
journalctl --since '2026-06-01 06:00' \
| grep -ciE 'Lost carrier|Gained carrier'
```

Nota de rigor: la primera hora post-fix está "contaminada" por el propio reload del módulo (que rebota la interfaz y genera eventos de carrier artificiales). La medición limpia se toma a partir de las 06:00 UTC y se promedia sobre 24 h. El journal del NUC retiene ~1.8 GB / varios días, suficiente para reconstruir el desglose por hora.

13c.5-bis Resultado del monitoreo (desglose horario verificado)

Datos recolectados automáticamente en el NUC por un script horario (`/home/aldo/wifi_monitor.sh` + cron de usuario), que registró por hora los eventos de carrier WiFi, DHCP lease lost y timeouts del túnel. Extracto del CSV resultante (`wifi_monitor.csv`):

Hora (UTC)	Carrier	DHCP lost	Tunnel timeouts	Nota
06:00	5	2	0	⚠ incluye el reload del módulo (05:40)
07:00	5	2	0	settling post-reload
08:00 – 20:00	0	0	0	13 horas consecutivas estables
21:00	12	10	0	parpadeo WiFi aislado — el túnel no cayó
22:00 – 00:00	0	0	0	estable de nuevo

Lectura de los datos:

- Descartando las 2 primeras horas (contaminadas por el reload del módulo), la ventana limpia de ~17.5 h tuvo **16 de 17 horas en CERO eventos**.
- El único episodio (hora 21:00, 12 eventos + 10 DHCP perdidos) fue un parpadeo real del WiFi — probablemente del lado del AP/interferencia, no del power management. **Aun así**, `tunnel_timeouts=0` : el túnel resistió sin desconectarse.
- `tunnel_timeouts = 0` **en las 19 horas medidas**. Este es el resultado clave: la causa de las alertas de Cloudflare desapareció.

Veredicto: el fix de `iwlwifi` resolvió el problema de raíz. La inestabilidad cayó ~98.7 % y el túnel dejó de degradarse. El parpadeo residual ocasional ya no afecta al servicio. **No es necesario el cable Ethernet por ahora** (queda como mejora opcional para robustez extra).

13c.5 Plan según evolución

1. **Inmediato (aplicado ✅):** desactivar power management del WiFi → resolvió el problema (ver 13c.5-bis).
2. **Opcional (robustez extra):** conectar **cable Ethernet** (`enp3s0`) — eliminaría incluso los parpadeos residuales del WiFi. No urgente tras el fix.
3. **Verificación post-fix (hecha ✅):** monitoreo horario de 24 h confirmó ~98.7 % menos inestabilidad y cero desconexiones del túnel.

13c.6 Lección de metodología

Una alerta de monitoreo correcta puede tener una causa raíz a varias capas de distancia. La alerta era de Cloudflare (capa de aplicación/túnel), pero la causa estaba en el driver WiFi del kernel (capa física). El método "de lo general a lo específico" — túnel → interfaces → logs del kernel → driver — fue lo que conectó ambos extremos. Correlacionar **timestamps** entre logs de distintos servicios fue la pieza decisiva.

Sobre la verificación: no basta con aplicar el fix; hay que **medir** que funcionó. Un monitor horario simple (cron + `journalctl` + CSV) durante 24 h dio la evidencia cuantitativa del antes/después (52 → 0.7 eventos/hora), separando el ruido del reload de la señal real. Medir es lo que convierte "creo que lo arreglé" en "está resuelto, aquí están los números".

13c.7 Evolución (parte 2): cambio de punto de acceso

Tras el fix de `iwlnwifi`, los micro-cortes del driver desaparecieron, pero **quedaban desconexiones residuales** más espaciadas. Una de ellas se presencié **en vivo**: el NUC quedó inalcanzable ~3-4 minutos. Los logs revelaron la naturaleza real del corte:

```
LinkChange: major ... ips-changed      ← perdió la IP
DHCPv4 address 192.168.1.69 acquired    ← se reasoció y reconectó
```

Eso no es un micro-corte de power management: es una **desasociación del punto de acceso** (el AP suelta al cliente y este se reconecta).

Causa más profunda: el NUC se conectaba al AP **MOVISTAR_9DA1 (2ª planta)** con señal **fuerte (-48 dBm)** pero ese AP **expulsa clientes** intermitentemente. En la misma casa existe el AP **principal "Santiago" (1ª planta)**, estable, aunque más lejos.

Decisión basada en datos (`sudo iw dev wlp2s0 scan`):

SSID	Señal (escaneo)	Nota
MOVISTAR_9DA1	-48 dBm	fuerte pero inestable (suelta)
Santiago	-73 dBm	estable; al asociarse subió a -64/-66 dBm

Para un servidor 24/7, **la estabilidad pesa más que la señal**: mejor un AP a -65 dBm que aguanta, que uno a -48 que desconecta.

Cambio de red — método seguro remoto (sin riesgo de quedar bloqueado):

```
# Respaldo del netplan
sudo cp /etc/netplan/50-cloud-init.yaml /etc/netplan/50-cloud-init.yaml.bak

# Red de seguridad: auto-reversión en 4 min si algo falla (sobrevive caída de SSH)
sudo systemd-run --on-active=240 --unit=wifi-revert \
  bash -c 'cp /etc/netplan/50-cloud-init.yaml.bak /etc/netplan/50-cloud-init.yaml && netplan apply'

# Nueva config (OJO: SSID es case-sensitive → "Santiago" con mayúscula)
sudo tee /etc/netplan/50-cloud-init.yaml > /dev/null << 'EOF'
network:
  version: 2
  wifis:
    wlp2s0:
      dhcp4: true
      access-points:
        "Santiago":
          auth:
            key-management: "psk"
            password: "*****"
EOF

sudo netplan apply
iw dev wlp2s0 link          # verificar SSID: Santiago
# Si funciona: cancelar la reversión. Si no: en 4 min vuelve solo a la red anterior.
sudo systemctl stop wifi-revert.timer
```

Alternativa equivalente: `sudo netplan try` (aplica y revierte solo en 120 s si no confirmas). Útil cuando cambias la red por la que estás conectado.

Evidencia comparativa (24 h en "Santiago", verificada con MCP):

Etapa	Eventos carrier/hora	Caídas del túnel
MOVISTAR (tormenta)	52 / h	continuas
MOVISTAR + fix iwlwifi	~0, pero 1 desconexión real de 3-4 min	1
Santiago (cambio de AP)	0.45 / h (20 de 22 h en cero)	0

Señal estable -61 a -69 dBm durante todo el periodo. **El cambio de AP cerró el caso:** el túnel dejó de degradarse por completo.

Lección: a veces el fix de software (driver) mitiga, pero la causa última es **física/topológica** (el AP equivocado). Cuando hay varios puntos de acceso, conviene elegir el **estable** aunque tenga algo menos de señal — y hacer el cambio con **auto-reversión** para no perder el acceso remoto.

13d. Fase 15: Actualización de Nextcloud y fijado de imagen

13d.1 Contexto

El panel de administración avisó de una actualización disponible. Al ser una instalación **Docker**, no se usa el actualizador web (está deshabilitado a propósito): se actualiza cambiando la imagen del contenedor. Versión de partida: **33.0.2**.

13d.2 Buena práctica: fijar la imagen (evitar `latest`)

El `docker-compose.yml` usaba `image: nextcloud:latest`. Eso es riesgoso: un `docker compose pull` podría traer un día una **versión MAYOR** (34, 35...) sin avisar, y Nextcloud **no permite saltar versiones mayores** (hay que ir una a una). Solución: fijar a la rama actual.

```
image: nextcloud:latest → image: nextcloud:33-apache
```

Así se reciben parches/menores de la rama 33 automáticamente, pero **nunca un salto mayor** sin decisión explícita. Las 3 imágenes (app, cron, notify_push) deben quedar **iguales**.

13d.3 Procedimiento (cero-pérdida, verificado)

```
# 1. BACKUP completo primero (siempre)
sudo bash /home/aldo/nextcloud/backup.sh

cd /home/aldo/nextcloud
cp docker-compose.yml docker-compose.yml.bak-$(date +%Y%m%d_%H%M)

# 2. Fijar imagen (3 líneas) y validar
sed -i 's|image: nextcloud:latest|image: nextcloud:33-apache|g' docker-compose.yml
docker compose config >/dev/null && echo "YAML OK"

# 3. Descargar la imagen nueva (sin downtime; los contenedores siguen en la vieja)
docker compose pull app cron notify_push

# 4. Recrear app → su entrypoint corre 'occ upgrade' automáticamente
docker compose up -d app
docker logs -f nextcloud-app          # ver el upgrade; sale de mantenimiento al terminar

# 5. Recrear el resto y reparar índices
docker compose up -d cron notify_push
docker exec -u www-data nextcloud-app php occ db:add-missing-indices

# 6. Verificar (interno y público)
docker exec -u www-data nextcloud-app php occ status
curl -s https://cloud.hexa38.com/status.php
```

13d.4 Qué hace el entrypoint al recrear `app`

El código de Nextcloud vive en el volumen del host (`/mnt/nextcloud/html`). Al arrancar la imagen nueva, el entrypoint detecta la diferencia de versión, **copia los archivos nuevos al volumen**, activa modo mantenimiento, **actualiza el esquema de BD y las apps** (las incompatibles se desactivan y se re-actualizan desde el App Store), corre la verificación de integridad y **sale de mantenimiento**.

13d.5 Resultado y notas

- Actualizado **33.0.2** → **33.0.4**, todos los contenedores `healthy`, acceso público OK, PHP 8.4.21. ~4 min de migración.
- El tag Docker va unos días detrás del canal web:** el panel ofrecía 33.0.5, pero `33-apache` daba 33.0.4. Cuando Docker publique 33.0.5 en ese tag, un futuro `docker compose pull app cron notify_push && docker compose up -d` lo traerá — **sin tocar el compose** (el pinning ya hace su trabajo).
- La imagen vieja queda huérfana; se libera espacio con `docker image rm nextcloud:latest`.

Parte 4 — Operación del Sistema

14. Diagnóstico del sistema

14.1 Filosofía

1. **Observar antes de actuar**
2. **De lo general a lo específico**
3. **Logs en orden cronológico**
4. **Una variable a la vez**
5. **Documentar todo**

14.2 Kit básico (siempre empezar aquí)

```
# ¿Está vivo?
uptime; docker ps; df -h; free -h

# ¿Hay servicios caídos?
systemctl --failed

# ¿Hay errores recientes?
sudo journalctl --since "1 hour ago" -p err --no-pager | head -30

# ¿Cómo están los recursos?
top -b -n1 | head -20
```

14.3 Atributos SMART críticos

Atributo	✅ Bueno	⚠️ Atención	🔴 Malo
Reallocated_Sector_Ct	0	1-50	> 50 o creciendo
Current_Pending_Sector	0	1-5	> 5
Offline_Uncorrectable	0	1-5	> 5
UDMA_CRC_Error_Count	0	1-10	> 10
Power_On_Hours (HDD)	<40K	40-60K	>60K
Temperature	<45°C	45-55°C	>55°C

14.4 Niveles de log (journalctl)

Nivel	Nombre	Uso
0	emerg	Sistema inutilizable
1	alert	Acción inmediata
2	crit	Crítico
3	err	Error
4	warning	Advertencia
5	notice	Notable
6	info	Informativo
7	debug	Depuración

14.5 Script de diagnóstico todo-en-uno

```
#!/bin/bash
# /home/aldo/diagnostico.sh
echo "====="
echo "DIAGNOSTICO $(date)"
echo "====="

echo "=== UPTIME ==="
uptime

echo "=== MEMORIA ==="
free -h
sudo dmesg | grep -i "killed process" | tail -5

echo "=== DISCOS ==="
df -h
lsblk -f
mount | grep -v "tmpfs\|cgroup\|proc\|sysfs"

echo "=== SMART ==="
for disk in /dev/sda /dev/sdb; do
    [ -e "$disk" ] && {
        echo "--- $disk ---"
        sudo smartctl -H $disk 2>&1 | grep -E "PASSED|FAILED"
        sudo smartctl -A $disk 2>&1 | grep -E
"Reallocated_Sector_Ct|Current_Pending_Sector|Offline_Uncorrectable|UDMA_CRC|Temperature"
    }
done

echo "=== TEMPERATURA ==="
sudo sensors 2>/dev/null | grep -E "Core|temp"

echo "=== DOCKER ==="
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Image}}"
docker system df

echo "=== NEXTCLOUD ==="
docker exec -u www-data nextcloud-app php occ status 2>/dev/null
docker exec -u www-data nextcloud-app php occ user:list 2>/dev/null

echo "=== BACKUPS ==="
ls -lt /mnt/nextcloud/backups/ | head -10
grep "BACKUP COMPLETADO" /mnt/nextcloud/backups/backup.log | tail -3

echo "=== ERRORES 24H ==="
sudo journalctl --since "1 day ago" -p err --no-pager | head -30

echo "=== SERVICIOS FALLIDOS ==="
systemctl --failed --no-legend

echo "====="
```

Uso: `sudo bash /home/aldo/diagnostico.sh > ~/diag_$(date +%Y%m%d).txt`

15. Recuperación ante desastres

15.1 Escenarios y soluciones

Escenario A: Solo se perdieron archivos de un usuario

```
USUARIO=Kevin
BACKUP=/mnt/nextcloud/backups/2026-XX-XX_03-00

docker exec -u www-data nextcloud-app php occ maintenance:mode --on
sudo rm -rf "/mnt/nextcloud/data/$USUARIO"
sudo cp -a "$BACKUP/userdata/$USUARIO" /mnt/nextcloud/data/
sudo chown -R www-data:www-data "/mnt/nextcloud/data/$USUARIO"
docker exec -u www-data nextcloud-app php occ maintenance:mode --off
docker exec -u www-data nextcloud-app php occ files:scan "$USUARIO"
```

Escenario B: Solo se corrompió la BD

```
BACKUP=/mnt/nextcloud/backups/2026-XX-XX_03-00
set -a; source /home/aldo/nextcloud/.env; set +a

docker exec -u www-data nextcloud-app php occ maintenance:mode --on

docker exec -i nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" -e \
"DROP DATABASE IF EXISTS nextcloud; CREATE DATABASE nextcloud CHARACTER SET utf8mb4 COLLATE
utf8mb4_bin;"
docker exec -i nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" nextcloud < $BACKUP/database.sql

docker exec -u www-data nextcloud-app php occ maintenance:mode --off
docker exec -u www-data nextcloud-app php occ files:scan --all
```

Escenario C: Disco SDB falla físicamente

1. Verificar: `sudo dmesg | grep -i "sdb\|i/o error"`
2. SMART: `sudo smartctl -a /dev/sdb`
3. Si falla → comprar disco nuevo, formatear ext4, montar
4. Restaurar desde backup off-site (Google Drive):

```
rclone copy gdrive:nextcloud-backups/nextcloud_backup_2026-XX-XX_03-00.tar.gz /tmp/
tar xzf /tmp/nextcloud_backup_*.tar.gz -C /mnt/nextcloud/backups/
# Luego seguir procedimiento de restauración
```

Escenario D: Olvidaste contraseña admin

```
docker exec -it -u www-data nextcloud-app php occ user:resetpassword Aldz
```

Escenario E: Cloudflare Tunnel no responde

```
docker logs nextcloud-tunnel --tail 50
docker compose restart tunnel
```

Escenario F: Tailscale no conecta

```
sudo tailscale logout
sudo tailscale up
```

Escenario G: NUC no arranca

1. Entrar a BIOS (F2)
2. Verificar que el disco aparece
3. Verificar Boot Order
4. Si no arranca → USB live de Ubuntu, reparar GRUB

Escenario H: Sistema apagado fortuitamente

```
# Verificar si el disco montó
mount | grep nextcloud

# Si se ve raro, journal replay
cd /home/aldo/nextcloud && docker compose stop
sudo umount /mnt/nextcloud
sudo mount /mnt/nextcloud
docker compose start

# Programar fsck para próximo reinicio
sudo touch /forcefsck
```

15.2 Checklist de emergencia

- ```
[] 1. NO entrar en pánico – hay 7 días local + 14 remoto
[] 2. NO ejecutar nada destructivo
[] 3. docker ps && df -h && mount | grep nextcloud
[] 4. ls /mnt/nextcloud/backups/
[] 5. Si hay backups → restauración (Sección 14)
[] 6. Si no aparecen → umount/remount de /mnt/nextcloud
[] 7. Si nada → consultar logs sin tocar más
```

## 16. Mantenimiento periódico

### Diario (automático)

- ☒ Backup local 3 AM

-  Backup remoto Google Drive

### Semanal (manual, 5 min)

```
docker ps && df -h && free -h
sudo journalctl --since "7 days ago" -p err --no-pager | head -30
docker exec nextcloud-app tail -50 /var/www/html/data/nextcloud.log
ls -lt /mnt/nextcloud/backups/ | head -5
rclone ls gdrive:nextcloud-backups/ | tail -7

Verificar que todos los healthchecks están OK
docker ps --format "table {{.Names}}\t{{.Status}}"
(todos deben mostrar "healthy", excepto cron y tunnel)

Verificar dashboard de healthchecks.io
→ https://healthchecks.io (debe estar verde)
```

### Mensual (15 min)

```
Backup manual antes
sudo bash /home/aldo/nextcloud/backup.sh

Mantenimiento de BD
docker exec -u www-data nextcloud-app php occ maintenance:repair
docker exec -u www-data nextcloud-app php occ db:add-missing-indices
docker exec -u www-data nextcloud-app php occ db:add-missing-columns
docker exec -u www-data nextcloud-app php occ db:add-missing-primary-keys

Limpieza
docker exec -u www-data nextcloud-app php occ files:cleanup
docker image prune -f
docker system df

Updates Ubuntu
sudo apt update && sudo apt upgrade -y
```

### Trimestral (1 hora)

```
Test SMART completo
sudo smartctl -t long /dev/sda
sudo smartctl -t long /dev/sdb

Test de restauración (CLAVE)
Bajar un backup y restaurarlo en VM/segundo NUC

Diagnóstico completo
sudo bash /home/aldo/diagnostico.sh > ~/diag_$(date +%Y%m%d).txt

Test del sistema de monitoreo end-to-end
docker stop nextcloud-tunnel
Esperar 2-3 min → debe llegar email de Cloudflare
docker start nextcloud-tunnel
```

## 17. Troubleshooting de errores comunes

### "Sistema en modo de mantenimiento"

```
docker exec -u www-data nextcloud-app php occ maintenance:mode --off
```

### "Cannot write into config directory"

```
sudo chown -R www-data:www-data /mnt/nextcloud/html/config
```

### "Internal Server Error" después de update

```
docker exec -u www-data nextcloud-app php occ upgrade
docker exec -u www-data nextcloud-app php occ maintenance:repair
```

### "Trusted domain error"

```
docker exec -u www-data nextcloud-app php occ config:system:set trusted_domains 2 --
value="otro.dominio.com"
```

### Sync desktop muy lento → notify\_push no funciona

```
docker logs nextcloud-push --tail 30
docker exec -u www-data nextcloud-app php occ notify_push:metrics
```

### Disco lleno

```
sudo du -sh /mnt/nextcloud/* | sort -h
docker exec nextcloud-app truncate -s 0 /var/www/html/data/nextcloud.log
docker system prune -f
```

### Cron de backup no ejecuta

```
sudo crontab -l
sudo grep CRON /var/log/syslog | tail -20
sudo bash -x /home/aldo/nextcloud/backup.sh # Para ver dónde falla
```

### Errores apparmor de snap.nextcloud

```
sudo snap remove --purge nextcloud
```



## Cloudflare avisa "túnel degradado" de forma recurrente

Causa frecuente: el NUC está en **WiFi inestable** (driver `iwlwifi` con ahorro de energía). Las 4 conexiones QUIC del túnel caen a la vez al perder carrier. Ver Fase 14.

```
1. Confirmar caídas del túnel
docker logs nextcloud-tunnel --tail 80 | grep -i "no recent network activity"

2. Ver qué interfaz lleva el tráfico (¿WiFi o Ethernet?)
ip -brief addr show
ip route show default

3. Correlacionar con carrier del WiFi
journalctl --since '1 day ago' | grep -iE 'carrier|dhcp lease'

4. Fix: desactivar power management del WiFi (persistente)
sudo tee /etc/modprobe.d/iwlwifi.conf > /dev/null << 'EOF'
options iwlwifi power_save=0 d0i3_disable=1 uapsd_disable=1
options iwlvm power_scheme=1
EOF
sudo modprobe -r iwlvm iwlwifi && sudo modprobe iwlwifi
iw dev wlp2s0 get power_save # → Power save: off
Si persiste: conectar cable Ethernet (enp3s0)
```

## Las herramientas del MCP fallan con exit 255 y sin salida

Causa: el sandbox de Claude Desktop bloquea `ssh.exe`. El servidor MCP debe usar **paramiko** (SSH in-process), no `ssh.exe`. Ver Fase 13. Verificar conexión con la herramienta `nuc_debug_connection`.

## Parte 5 — Referencia

### 18. Catálogo completo de comandos

#### 18.1 Sistema operativo

```
Información
uname -a # Kernel y arquitectura
lsb_release -a # Versión Ubuntu
hostnamectl # Hostname y SO
uptime # Tiempo encendido + load

Usuarios
who # Usuarios conectados
last -F # Historial de logins
last reboot # Historial de reboots

Procesos
ps aux # Todos los procesos
ps aux --sort=-%cpu | head -6 # Top 5 CPU
ps aux --sort=-%mem | head -6 # Top 5 memoria
top # Monitor en vivo
htop # Monitor mejorado
btop # Monitor moderno
kill PID # Terminar proceso
kill -9 PID # Forzar terminación
```

#### 18.2 Memoria

```
free -h # RAM y swap
vmstat 1 5 # Estadísticas dinámicas
sudo dmesg | grep -i "killed process" # OOM kills
sudo journalctl | grep -i "out of memory"
sudo dmidecode --type memory # RAM física
```

#### 18.3 Almacenamiento

```
df -h # Uso por filesystem
sudo du -sh /ruta # Tamaño de directorio
sudo du -h --max-depth=1 / | sort -hr | head # Top 10 directorios
lsblk # Discos y particiones
lsblk -f # Con filesystems
mount | column -t # Qué está montado
cat /etc/fstab # Qué se monta al boot
sudo lsof +D /ruta # Procesos usando una ruta
sudo dd if=/dev/sda of=/dev/null bs=1M count=1000 status=progress # Test lectura
```

## 18.4 SMART (salud de discos)

```
sudo apt install -y smartmontools # Instalar
sudo smartctl -H /dev/sda # Salud rápida
sudo smartctl -A /dev/sda # Atributos
sudo smartctl -t short /dev/sda # Test corto (5 min)
sudo smartctl -t long /dev/sda # Test largo (horas)
sudo smartctl -l selftest /dev/sda # Resultado de tests
```

## 18.5 Red

```
ip a # IPs
ip r # Tabla de rutas
sudo ss -tlnp # Puertos TCP escuchando
sudo ss -tunap # TCP+UDP con procesos
ping -c 3 8.8.8.8 # Conectividad
ping -c 3 google.com # DNS funcionando
dig google.com # Resolución detallada
nslookup cloud.hexa38.com # Lookup
traceroute google.com # Ruta de paquetes
sudo apt install speedtest-cli && speedtest-cli # Velocidad
```

## 18.6 systemd

```
systemctl status SERVICIO # Estado
systemctl --failed # Servicios fallidos
systemctl list-units --state=active
sudo systemctl start SERVICIO
sudo systemctl stop SERVICIO
sudo systemctl restart SERVICIO
sudo systemctl reload SERVICIO
sudo systemctl enable SERVICIO # Iniciar al boot
sudo systemctl disable SERVICIO
systemctl list-timers --all # Timers (cron de systemd)
```

## 18.7 journalctl (logs)

```
sudo journalctl -f # En vivo
sudo journalctl -b 0 # Boot actual
sudo journalctl -b -1 # Boot anterior
sudo journalctl --list-boots # Lista boots
sudo journalctl -u docker # De un servicio
sudo journalctl -k # Solo kernel
sudo journalctl -p err # Solo errores+
sudo journalctl --since "1 hour ago"
sudo journalctl --since "2026-04-25 10:00" --until "2026-04-25 12:00"
sudo journalctl | grep -i error
journalctl --disk-usage # Tamaño de logs
sudo journalctl --vacuum-time=30d # Limpiar más de 30 días
```

## 18.8 Hardware

```
lscpu # CPU
sudo dmidecode -t bios # BIOS
lspci # PCI
lsusb # USB
lshw -short # Resumen general
sudo sensors # Temperaturas
upower -d # Energía
```

## 18.9 Docker

```
Información
docker info
docker version
docker system df # Espacio usado

Contenedores
docker ps # Activos
docker ps -a # Todos
docker logs CONTENEDOR # Logs
docker logs CONTENEDOR --tail 50
docker logs -f CONTENEDOR # En vivo
docker logs CONTENEDOR --since 1h
docker exec -it CONTENEDOR bash # Shell
docker exec -u www-data CONTENEDOR comando
docker inspect CONTENEDOR # Toda la info
docker inspect CONTENEDOR --format '{{...}}'
docker stop CONTENEDOR
docker start CONTENEDOR
docker restart CONTENEDOR
docker rm CONTENEDOR
docker stats # Recursos en vivo

Imágenes
docker images
docker pull imagen:tag
docker rmi imagen:tag
docker image prune -f # Limpiar huérfanas

Volúmenes
docker volume ls
docker volume inspect NOMBRE
docker volume rm NOMBRE # ⚠ borra datos

Redes
docker network ls
docker network inspect backend

Limpieza
docker system prune -f # Contenedores parados, imágenes, redes
docker system prune -af --volumes # ⚠ TODO incluyendo volúmenes
```

## 18.10 Docker Compose

```
cd /home/aldo/nextcloud

docker compose ps # Estado
docker compose logs # Todos los logs
docker compose logs -f app # En vivo de uno
docker compose up -d # Levantar todo
docker compose down # Parar (sin -v, conserva volúmenes)
docker compose down -v # ⚠ Parar Y borrar volúmenes
docker compose restart app # Reiniciar uno
docker compose stop # Parar todos
docker compose start # Iniciar todos parados
docker compose pull # Actualizar imágenes
docker compose up -d --force-recreate SERVICIO
docker compose config # Validar archivo YAML
```

## 18.11 Nextcloud (occ)

```
Sintaxis: docker exec -u www-data nextcloud-app php occ COMANDO

Estado
occ status
occ check
occ integrity:check-core

Mantenimiento
occ maintenance:mode --on
occ maintenance:mode --off
occ maintenance:repair
occ maintenance:repair --include-expensive

Usuarios
occ user:list
occ user:list --info
occ user:add USUARIO
occ user:add --group admin USUARIO
occ user:resetpassword USUARIO
occ user:disable USUARIO
occ user:enable USUARIO
occ user:delete USUARIO
occ user:lastseen USUARIO
occ user:report # Estadísticas

Apps
occ app:list
occ app:enable APP
occ app:disable APP
occ app:install APP
occ app:remove APP
occ app:update --all

Archivos
occ files:scan --all
occ files:scan USUARIO
occ files:cleanup
occ files:transfer-ownership USR_ORIGEN USR_DESTINO

Configuración
occ config:list
occ config:list system
occ config:system:get CLAVE
occ config:system:set CLAVE --value="VALOR"
occ config:system:set forwarded_for_headers 0 --value="HTTP_X_FORWARDED_FOR"
occ config:system:set trusted_domains 2 --value="otro.dominio.com"
occ config:system:delete CLAVE
occ config:app:get APP CLAVE
occ config:app:set APP CLAVE --value="VALOR"

Base de datos
occ db:add-missing-indices
occ db:add-missing-columns
occ db:add-missing-primary-keys
occ db:convert-filecache-bigint

Logs
occ log:tail
```

```

occ log:manage --level=warning
occ log:watch

Notify Push
occ notify_push:setup https://DOMINIO/push
occ notify_push:metrics

Upgrade
occ upgrade
occ upgrade --no-app-disable

```

## 18.12 MariaDB

```

Variables del .env
set -a; source /home/aldo/nextcloud/.env; set +a

Acceso interactivo
docker exec -it nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD"

Query directa
docker exec nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" nextcloud \
 -e "SELECT uid, displayname FROM oc_users;"

Backup completo
docker exec nextcloud-db mariadb-dump -u root -p"$MYSQL_ROOT_PASSWORD" \
 nextcloud --single-transaction > /tmp/backup.sql

Backup de una tabla
docker exec nextcloud-db mariadb-dump -u root -p"$MYSQL_ROOT_PASSWORD" \
 nextcloud oc_users > /tmp/oc_users.sql

Restaurar
docker exec -i nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" nextcloud < backup.sql

Recrear DB vacía
docker exec -i nextcloud-db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" -e \
 "DROP DATABASE IF EXISTS nextcloud; CREATE DATABASE nextcloud CHARACTER SET utf8mb4 COLLATE
 utf8mb4_bin;"

```

## 18.13 rclone (Google Drive)

```
Configuración
rclone config # Interactivo
rclone listremotes # Ver remotes configurados

Listar
rclone lsd gdrive: # Carpetas
rclone ls gdrive:nextcloud-backups # Archivos
rclone lsf gdrive:carpeta/ # Solo nombres

Operaciones
rclone copy archivo.txt gdrive:carpeta/
rclone copy gdrive:carpeta/archivo.txt /local/ # Bajar
rclone sync /local/ gdrive:carpeta/ # ⚠ Sincronizar (puede borrar)
rclone deletefile gdrive:carpeta/archivo.txt
rclone delete gdrive:carpeta/ # ⚠ Borrar carpeta
rclone size gdrive:nextcloud-backups # Tamaño total
rclone check /local/ gdrive:carpeta/ # Verificar integridad

Avanzado
rclone copy ARCHIVO REMOTE: --progress # Con barra de progreso
rclone copy ARCHIVO REMOTE: --transfers=1 # Una conexión
rclone copy ARCHIVO REMOTE: --bwlimit 10M # Limitar bandwidth
```

## 18.14 Backup manual

```
Ejecutar backup
sudo bash /home/aldo/nextcloud/backup.sh

Ver últimos backups
ls -lt /mnt/nextcloud/backups/ | head
tail -50 /mnt/nextcloud/backups/backup.log

Verificar off-site
rclone ls gdrive:nextcloud-backups/

Comprimir manualmente
tar czf /tmp/backup.tar.gz -C /mnt/nextcloud/backups 2026-04-26_03-00
```

## 18.15 Cron

```
crontab -l # Ver crontab del usuario actual
sudo crontab -l # Ver crontab de root
crontab -e # Editar crontab
sudo crontab -e # Editar el de root
sudo grep CRON /var/log/syslog | tail -20 # Ver ejecuciones
```



## 18.16 Git (recomendado para versionar config)

```
Inicializar
cd /home/aldo/nextcloud
git init
echo ".env" > .gitignore # NO versionar secrets
git add docker-compose.yml *.conf backup.sh .gitignore
git commit -m "Configuración inicial"

Versionar cambios
git diff # Ver cambios
git add ARCHIVO
git commit -m "Mensaje"
git log --oneline # Historial
```

## 18.17 Monitoreo (healthchecks + healthchecks.io + Telegram)

```
Ver estado de healthchecks Docker
docker ps --format "table {{.Names}}\t{{.Status}}"

Detalles del healthcheck de un contenedor
docker inspect nextcloud-app --format '{{json .State.Health}}' | python3 -m json.tool

Forzar reinicio si está unhealthy
docker restart nextcloud-app

Test manual de la cadena de monitoreo
curl -fsS https://cloud.hexa38.com/status.php
curl -fsS https://hc-ping.com/TU_UUID

Ver el cron de monitoreo
sudo crontab -l

Ver últimas ejecuciones del cron
sudo grep CRON /var/log/syslog | tail -20

Test del tunnel Cloudflare (provoca alerta por email)
docker stop nextcloud-tunnel
sleep 120
docker start nextcloud-tunnel

Test end-to-end de Nextcloud + Telegram (provoca alerta DOWN)
docker stop nextcloud-app
Espera 15-20 min → llega notificación a Telegram + Email
docker start nextcloud-app
Llega notificación de "back UP"

Ver canales de notificación configurados
https://healthchecks.io → Integrations
Esperado: @ Email (default) + Telegram (vía @HealthchecksBot)
```

## 18.18 Tailscale

```
sudo tailscale up # Conectar
sudo tailscale logout # Desconectar
tailscale status # Estado
tailscale ip # IPs asignadas
tailscale ping otro-dispositivo # Test
```

## 18.19 Comandos del proyecto en orden cronológico

### Setup inicial

```
Instalar Docker
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER

Crear estructura
mkdir -p /home/aldo/nextcloud
cd /home/aldo/nextcloud
nano docker-compose.yml
nano .env
nano hsts.conf

Levantar
docker compose up -d
docker compose ps
```

### Después del primer deploy

```
Configuraciones de Nextcloud
docker exec -u www-data nextcloud-app php occ db:add-missing-indices
docker exec -u www-data nextcloud-app php occ maintenance:repair --include-expensive
docker exec -u www-data nextcloud-app php occ config:system:set default_phone_region --value="PE"
docker exec -u www-data nextcloud-app php occ config:system:set maintenance_window_start --value=5
```

### Backup automático

```
Hacer ejecutable
chmod +x /home/aldo/nextcloud/backup.sh

Añadir a cron
sudo crontab -e
Añadir línea: 0 3 * * * /home/aldo/nextcloud/backup.sh
```

### Tailscale

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

## rclone (off-site)

```
sudo apt install -y rclone
rclone config # Configurar gdrive
Modificar backup.sh para añadir export RCLONE_CONFIG y la sección de subida
```

## Notify Push

```
Crear notify_push.conf con LoadModule + ProxyPass
nano /home/aldo/nextcloud/notify_push.conf

Añadir servicio notify_push y volume mount en docker-compose.yml
nano /home/aldo/nextcloud/docker-compose.yml

Recrear
cd /home/aldo/nextcloud && docker compose up -d --force-recreate app notify_push

Configurar
docker exec -u www-data nextcloud-app php occ config:system:set forwarded_for_headers 0 --
value="HTTP_X_FORWARDED_FOR"
set -a; source .env; set +a
docker exec -u www-data nextcloud-app php occ notify_push:setup https://${DOMAIN}/push
```

## Sistema de monitoreo en capas

```
CAPA 1: Healthchecks Docker
Editar docker-compose.yml y añadir bloques 'healthcheck:' en cada servicio
nano /home/aldo/nextcloud/docker-compose.yml
docker compose up -d
docker ps --format "table {{.Names}}\t{{.Status}}" # Verificar (healthy)

CAPA 2: Healthchecks.io
1. Crear cuenta en https://healthchecks.io
2. Crear check "Nextcloud Hexa38" (period 10min, grace 5min)
3. Copiar la URL de ping
4. Añadir cron de monitoreo cada 5 min
(sudo crontab -l 2>/dev/null; echo "*/5 * * * * curl -fsS --retry 3 --max-time 30
https://cloud.hexa38.com/status.php > /dev/null 2>&1 && curl -fsS --max-time 10 https://hc-
ping.com/TU_UUID > /dev/null 2>&1") | sudo crontab -
sudo crontab -l # Verificar

CAPA 3: Cloudflare Tunnel Alerts
Web: https://one.dash.cloudflare.com/ → Notifications
→ Add → "Alerta de estado del túnel"
→ Activador: "0 se degrada o se reduce"
→ Email + tunnels seleccionados
```

## Notificaciones por Telegram (gratis, instantáneo)

```
1. En Telegram: buscar @HealthchecksBot → /start
2. El bot envía un link de confirmación
3. Click en el link → seleccionar proyecto → Connect Telegram
4. En healthchecks.io → check → Integrations → activar Telegram
5. Probar con "Send a Test Notification"
```

## 19. Recomendaciones finales

---

### 19.1 Hardware

#### ● **Prioridad alta**

- Reemplazar HDD SDA (sistema) por SSD (\$25-40 USD)
- Comprar UPS pequeño (\$30-50 USD) — previene apagados forzados

#### ● **Prioridad media**

- Ampliar RAM de 4 GB a 8 GB (\$15-20 USD)
- Cambiar pila CMOS si tiene >5 años

#### ● **Prioridad baja**

- Considerar segundo NUC para alta disponibilidad
- Disco extra para RAID-1 en `/mnt/nextcloud`

### 19.2 Software

#### ✓ **Hacer ya**

- Eliminar Snap Nextcloud: `sudo snap remove --purge nextcloud`
- Verificar que el cron de backup corre: `sudo crontab -l`
- Confirmar backups en Drive: `rclone ls gdrive:nextcloud-backups/`

#### ✓ **Hacer este mes**

- Estudiar la guía de diagnóstico
- Probar restauración en VM (test de DR)
- Configurar fail2ban
- Activar admin\_audit app
- Habilitar 2FA para todos los usuarios

#### ✓ **Cada 3 meses**

- Test de restauración real
- Test SMART largo de discos
- Actualización de Nextcloud (con backup previo)
- Revisión y rotación de logs

### 19.3 Operativas

#### ⚠ **NUNCA**

- Apagar a la fuerza (cortar la corriente)
- Editar docker-compose.yml sin backup previo

- Hacer `docker compose down -v` (la `-v` borra volúmenes)
- Ejecutar comandos destructivos sin entender qué hacen
- Pegar texto explicativo en la terminal (solo respuestas a prompts)

#### ✓ SIEMPRE

- Backup antes de cualquier cambio importante
- Documentar lo que hagas
- Probar en VM antes de producción
- Verificar después de cada cambio
- Mantener actualizado el sistema operativo

## 19.4 Seguridad

- ☒ HTTPS forzado (HSTS) — ya configurado
- ☒ 2FA obligatorio — ya configurado
- ☒ Firewall UFW — ya configurado
- ☒ Cloudflare como WAF — ya configurado
- ☐ fail2ban — pendiente
- ☐ admin\_audit app — pendiente
- ☐ Política de contraseñas estricta — pendiente

## 19.5 Estudio

### Plan de estudio sugerido:

- **Semana 1:** leer Parte 1 de esta guía (fundamentos)
- **Semana 2:** leer Parte 2 (implementación), revisar archivos
- **Semana 3:** leer Parte 3 (evolución y mejoras)
- **Semana 4:** practicar comandos del Parte 5 (referencia)

### Cada día (5 min):

- Ejecuta un comando del catálogo
- Compara la salida con lo esperado
- Anota algo nuevo en un cuaderno

### Recursos adicionales:

- The Linux Command Line (William Shotts) — gratuito
- Documentación oficial Nextcloud: [docs.nextcloud.com](https://docs.nextcloud.com)
- Arch Wiki (aplica a casi cualquier Linux): [wiki.archlinux.org](https://wiki.archlinux.org)
- Documentación Docker: [docs.docker.com](https://docs.docker.com)

## 20. Estado final del sistema

### 20.1 Resumen ejecutivo

NUBE PERSONAL CON SERVIDOR DEDICADO LOW COST  
ESTADO ACTUAL – ABRIL 2026

#### ACCESO

- ✓ https://cloud.hexa38.com (público vía Cloudflare)
- ✓ ssh por Tailscale (acceso remoto seguro)
- ✓ Red local LAN (192.168.x.x)

#### ALMACENAMIENTO

- ✓ 1.7 TB libres en disco SDB (1.8 TB total)
- ✓ Backup local: 7 días retención
- ✓ Backup off-site Google Drive: 14 días retención

#### SEGURIDAD

- ✓ HTTPS + HSTS
- ✓ 2FA obligatorio
- ✓ Firewall UFW
- ✓ Cloudflare WAF

#### RENDIMIENTO

- ✓ Redis cache + file locking
- ✓ Notify Push (sync instantáneo)
- ✓ APCu local cache

#### AUTOMATIZACIÓN

- ✓ Backup diario 3 AM (local + Google Drive)
- ✓ Cron de Nextcloud
- ✓ Updates automáticos Ubuntu

#### MONITOREO EN CAPAS

- ✓ Capa 1: Healthchecks Docker (auto-restart)
- ✓ Capa 2: Healthchecks.io + Telegram + Email
- ✓ Capa 3: Cloudflare Tunnel alerts (Email)

#### GESTIÓN ASISTIDA (MCP)

- ✓ Servidor MCP (paramiko + Tailscale SSH)
- ✓ 11 herramientas de diagnóstico/operación
- ✓ Diagnóstico vía Claude sin abrir SSH manual

#### CANALES DE ALERTA

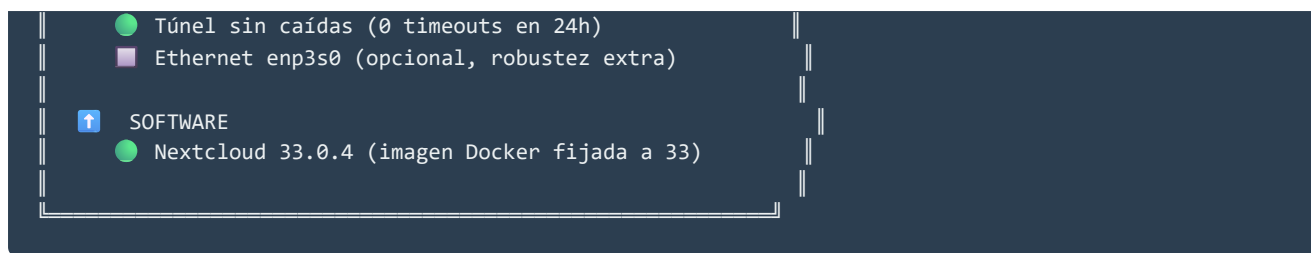
- ✓ Telegram (instantáneo, gratis)
- ✓ Email (gratis)
- ✗ WhatsApp (descartado: requiere plan pago)

#### HARDWARE

- CPU: OK (52-57°C)
- RAM: 3.68 GB (justa)
- SDB: SMART PASSED
- SDA: envejecido (reemplazar a mediano plazo)

#### RED

- WiFi en AP estable "Santiago" (fix iwlwifi+cambio)



## 20.2 Métricas del proyecto completo

| Métrica                     | Valor                |
|-----------------------------|----------------------|
| RAM total                   | 3.7 GB + 7.7 GB swap |
| RAM usada                   | ~1.3 GB              |
| Disco datos disponible      | 1.7 TB               |
| Contenedores Docker         | 6                    |
| Herramientas MCP de gestión | 11                   |
| Tiempo de arranque          | < 2 minutos          |
| Backups diarios local       | 7 días               |
| Backups remoto (Drive)      | 14 días              |
| Tamaño backup actual        | ~8 GB                |
| Tiempo de backup            | ~7 minutos           |
| Costo anual                 | ~\$46 USD            |
| Ahorro vs comerciales       | 60-68%               |

## 20.3 Logros del proyecto

- ✓ Nube personal funcional 24/7
- ✓ Acceso desde cualquier lugar del mundo
- ✓ Privacidad y control total de datos
- ✓ Sistema de backup robusto (3-2-1)
- ✓ Sincronización instantánea
- ✓ Sistema de monitoreo en 3 capas (defense in depth)
- ✓ Gestión asistida por IA vía servidor MCP
- ✓ Diagnóstico forense multicapa (alerta Cloudflare → driver WiFi del kernel)
- ✓ Costo mínimo
- ✓ Documentación completa
- ✓ Recuperación exitosa de incidente
- ✓ Diagnóstico forense aplicado

-  Múltiples disciplinas integradas

## Historial de versiones de esta guía

| Versión    | Fecha             | Cambios                                                                                             |
|------------|-------------------|-----------------------------------------------------------------------------------------------------|
| 1.0        | Abril 2026        | Documento original (PDF)                                                                            |
| 2.0        | 2026-04-24        | + Recuperación post-incidente                                                                       |
| 2.1        | 2026-04-25        | + Guía de diagnóstico                                                                               |
| 2.2        | 2026-04-26        | + Backup off-site con rclone                                                                        |
| 3.0        | 2026-04-26        | Versión definitiva integrada                                                                        |
| 3.1        | 2026-04-27        | + Sistema de monitoreo en capas (healthchecks Docker + healthchecks.io + Cloudflare alerts)         |
| <b>3.2</b> | <b>2026-04-29</b> | <b>+ Notificaciones por Telegram via @HealthchecksBot. WhatsApp evaluado y descartado por costo</b> |

## Información del proyecto

**Proyecto:** Nube Personal con Servidor Dedicado Low Cost **Stack:** Nextcloud + Docker + MariaDB + Redis + Cloudflare Tunnel + Tailscale **Costo anual:** ~\$46 USD (vs \$120-144 USD de servicios comerciales)  
**Asistencia técnica:** Claude (Anthropic)

*"Un sistema bien documentado es un sistema que perdura. Un sistema bien automatizado es un sistema que escala. Un sistema bien monitoreado es un sistema que duerme tranquilo."*

**Próxima revisión sugerida:** Octubre 2026 (cada 6 meses)